

COMP4610_01 - ADVANCED DATABASE SYSTEMS (WINTER 2026) DATA MINING PROJECT

Predicting Honeybee Health from Hive and Weather Data
Leveraging Colony Checklists, Hive Metrics, and Weather Patterns
to Forecast Bee Colony Health

Final Project Report

Group 5

Submitted by:

Bhavya Bhola T00729544

Kenji Lopez T00712309

Kshitij Santokhee T00736607

Nilam Rai T00688491

Jugias Girn T00712501

Submitted to:

Dr. Sukhchandan
Assistant Teaching Professor

INDEX

Section	Page
1. Introduction	3
2. Objectives	3
3. Problems Addressed	4
4. Dataset Overview	5
5. Data Preprocessing - Phase 1	6
6. Data Modelling - Phase 2	41
7. Model Results and Comparison	45
8. Best Model Analysis	47
9. Hyperparameter Tuning	48
10. Deep Learning Models	49
11. Key Findings	53
12. Future Work	54
13. Conclusion	54
14. Team Contribution	55
15. References	55

Notebook link:

<https://colab.research.google.com/drive/1foGKzz5GBEeUUSDooisfxlK-Rcqvl9Za?usp=sharing>

1. Introduction

1.1 Background

Honeybees are among the most important insects in the world when it comes to agriculture. They are responsible for pollinating a large share of the food crops that humans depend on. Over the past few decades, beekeepers and researchers have noticed a steady increase in colony losses across many regions. Colonies decline or collapse due to a combination of factors including poor nutrition, parasites, disease, queen problems, and unfavorable weather conditions. The challenge is that identifying an unhealthy colony early enough to intervene is difficult, especially when managing a large number of hives spread across multiple locations.

This project works with a real-world dataset from Kaggle titled "Predicting Honeybee Health from Hive and Weather Data." The dataset was collected from apiaries located in North Carolina and Utah, covering inspection records from February 2016 through August 2019. It includes hive inspection checklists, hive identification details, apiary location information, and weather station readings recorded on an hourly basis. By combining these sources, the dataset captures both the internal state of each colony and the external environmental conditions at the time of each inspection.

1.2 Purpose of the Project

The main purpose of this project is to analyze hive inspection records alongside environmental weather data in order to identify the patterns and conditions that are linked to unhealthy honeybee colonies. The project applies data mining techniques across two phases. Phase 1 covers data preprocessing - cleaning, integrating six relational tables, noise reduction, feature engineering, and exploratory data analysis. Phase 2 covers data modelling - training and comparing seven classification algorithms to determine which one best predicts whether a colony is healthy or not.

1.3 Relevance to Database Systems and Business Domain

From a database systems perspective, this project is a practical exercise in working with a relational dataset stored across multiple tables with defined relationships. The dataset requires proper integration using primary and foreign key joins before any analysis can be performed, which is a core challenge in real-world data environments where information is rarely stored in a single flat file.

From a business perspective, beekeeping is both a commercial and agricultural activity. Beekeepers who manage large numbers of hives need a way to predict which hives are at risk before they lose colonies. Early detection of unhealthy hives can reduce losses, lower operational costs, and improve overall colony management decisions. The ability to forecast hive health using inspection records and weather data has practical value for beekeeping operations, agricultural businesses that depend on pollination services, and environmental monitoring programs.

2. Objectives

2.1 Main Goal

The central goal of this project is to use data mining techniques to preprocess and analyze a multi-source honeybee dataset, and to train classification models that can predict whether a hive is healthy or not based on inspection attributes and weather conditions.

2.2 Specific Objectives

- Integrate six separate relational tables into one unified dataset that links colony health with hive details, apiary location, and environmental weather conditions.
- Perform thorough data cleaning by addressing missing values, removing duplicates, fixing inconsistent data types, and handling outliers.
- Apply noise reduction techniques such as equal-width binning, mean smoothing, rolling average smoothing, and regression smoothing to weather measurements.
- Aggregate hourly weather records into daily summaries that align with the periodic nature of hive inspections.
- Engineer new features including a Hive Stress Index, a Weather Stress Index, a Combined Colony Risk Score, and a Temperature Range feature.
- Conduct exploratory data analysis through fifteen visualizations covering distributions, correlations, seasonal trends, clustering, and feature importance.
- Apply feature selection and PCA to reduce dimensionality and prepare the dataset for modelling.
- Train and compare seven classification models to find the best predictor of colony health.
- Explore hyperparameter tuning using GridSearchCV and RandomizedSearchCV to improve model performance.
- Investigate deep learning approaches including a Feedforward Neural Network and an LSTM model for time-series inspection data.

2.3 Aims of the project

This project aims to demonstrate that colony health is not determined by a single factor but by a combination of inspection conditions and environmental variables. It also shows that raw multi-table data requires significant preprocessing before any meaningful modelling can be applied. The two-phase pipeline built in this project - from raw data to trained classifiers - provides a complete data mining workflow applicable to similar real-world prediction problems.

3. Problems Addressed

3.1 Technical Problems

Multi-table structure requiring integration: The dataset is split across six relational tables. None of these tables could be analyzed in isolation, requiring a multi-step merge process using ApiaryID, HiveID, StationID, ObsID, and Date as join keys.

Missing values: HCC_Inspections had 85 total missing values across Brood (16), Bees (16), Queen (16), Space (16), Stressors (15), Food (4), and InsptDate (2). Hourly_Weather had 154 missing values including Wind_Direction (26) and 16 missing values each across Temperature, Humidity, Pressure, and other columns. Hive_Information had 1 missing Hive_Tag entry.

Incorrect data types: Several weather columns were stored as object (text) type in the merged dataset, preventing numeric operations until explicitly converted.

Row explosion from one-to-many join: Merging inspection records with hourly weather data by date caused the dataset to expand from 2,404 inspection rows to 35,656 rows, because each inspection matched every hourly weather record on the same day. This had to be resolved through daily weather aggregation.

Class imbalance: Out of 2,404 inspection records, 1,531 (63.7%) are labeled unhealthy and only 873 (36.3%) are labeled healthy. This imbalance needed to be addressed before training classifiers.

Noisy weather measurements: Hourly weather readings contain natural short-term fluctuations that do not represent meaningful environmental changes. These required smoothing to produce more stable feature values.

High dimensionality after feature engineering: After merging and creating new features, the dataset had 66 feature columns. Many were redundant or non-predictive, requiring feature selection and dimensionality reduction.

3.2 Business Problems

Delayed detection of unhealthy colonies: Beekeepers typically inspect hives every one to two weeks. By the time a colony is visibly struggling, the decline may already be severe. A predictive tool could flag at-risk hives earlier.

Difficulty identifying which factors matter most: Beekeepers observe many attributes during an inspection. It is not always clear which factors carry the most weight in determining colony health. Data mining provides a way to quantify each factor's importance.

Geographic and environmental variability: The dataset covers apiaries in two distinct climates - North Carolina and Utah. Environmental conditions differ significantly between locations, making a data-driven approach more reliable than manual rules.

Operational costs of colony loss: Losing a colony represents lost pollination capacity, lost honey production, and equipment replacement costs. Predicting which colonies are heading toward failure allows beekeepers to allocate resources more efficiently.

4. Dataset Overview

The dataset consists of six relational CSV files downloaded from Kaggle. The tables below summarize each file and the missing value counts identified during data quality assessment. Dataset link: <https://www.kaggle.com/datasets/jocelyndumlao/predicting-honeybee-health-from-hive-and-weather/data>

Table 1: Dataset Summary

Table Name	Rows	Cols	Description
HCC_Inspections	2,404	11	Colony health checklist - one row per inspection. Target: Healthy (Yes/No)

Hive_Information	188	3	Hive tag identifiers linked to apiaries
Apiary_Information	13	4	Apiary names and locations (NC and UT)
Hourly_Weather	3,672	15	Hourly weather readings from 3 stations
Weather_Stations	3	3	Station details: Clayton, Durham (NC), Salt Lake City (UT)
Weather_Observations	1,776	4	Date and hour for each weather observation

Table 2: Missing Values by Table

Table	Total Missing	Affected Columns
HCC_Inspections	85	Brood(16), Bees(16), Queen(16), Space(16), Stressors(15), Food(4), InsptDate(2)
Hive_Information	1	Hive_Tag(1)
Apiary_Information	0	None
Hourly_Weather	154	Wind_Direction(26), Temperature(16), Humidity(16), all other weather cols(16 each)
Weather_Stations	0	None
Weather_Observations	0	None

The inspection records span from February 2016 to August 2019 covering 188 unique hives across 13 apiaries. The target variable Healthy is binary - 873 records are labeled Yes (36.3%) and 1,531 are labeled No (63.7%), confirming a class imbalance that must be handled before modelling.

5. Data Preprocessing - Phase 1

The preprocessing pipeline was carried out in a structured order. Each step is described below along with the exact outputs recorded from the notebook.

Pipeline Overview:

1. Dataset Exploration
2. Data Quality Assessment
3. Data Cleaning
4. Data Integration
5. Post-Integration Cleaning
6. Noisy Data Handling (by Binning & Smoothing)
7. Feature Aggregation
8. Feature Engineering
9. Exploratory Data Analysis & Visualizations
10. Feature Encoding & Normalization
11. Outlier Detection
12. Extracting X and y as independent variables
13. Data Reduction (using Feature Selection + PCA)

5.1 Data Quality Assessment

Before making any changes to the data, a quality check was performed on all six tables. Missing values were counted per column, duplicate records were checked, and missing value patterns were visualized using heatmaps. The assessment confirmed that no duplicates existed in any table (all tables returned 0 duplicates). Missing values were found in HCC_Inspections and Hourly_Weather as detailed in Table 2 above.

```

4. Data Quality Assessment

4.1 Checking Missing Values

[]
print('Missing values in each table:\n')
for name, df in datasets.items():
    missing = df.isnull().sum()
    total_missing = missing.sum()
    print(f'--- {name} (total missing: {total_missing}) ---')
    print(missing[missing > 0] if total_missing > 0 else ' No missing values')
    print()

Missing values in each table:

--- HCC_Inspections (total missing: 85) ---
InsptDate      2
Brood          16
Bees           16
Queen          16
Food           4
Stressors      15
Space          16
dtype: int64

--- Hive_Information (total missing: 1) ---
Hive_Tag       1
dtype: int64

--- Apiary_Information (total missing: 0) ---
No missing values

--- Hourly_Weather (total missing: 154) ---
Temperature     16
Humidity         16
Dew_Point       16
Wind_Direction  26
Wind_Speed      16
Wind_Gust       16
Pressure        16
Precip          16
Condition       16
dtype: int64

--- Weather_Stations (total missing: 0) ---
No missing values

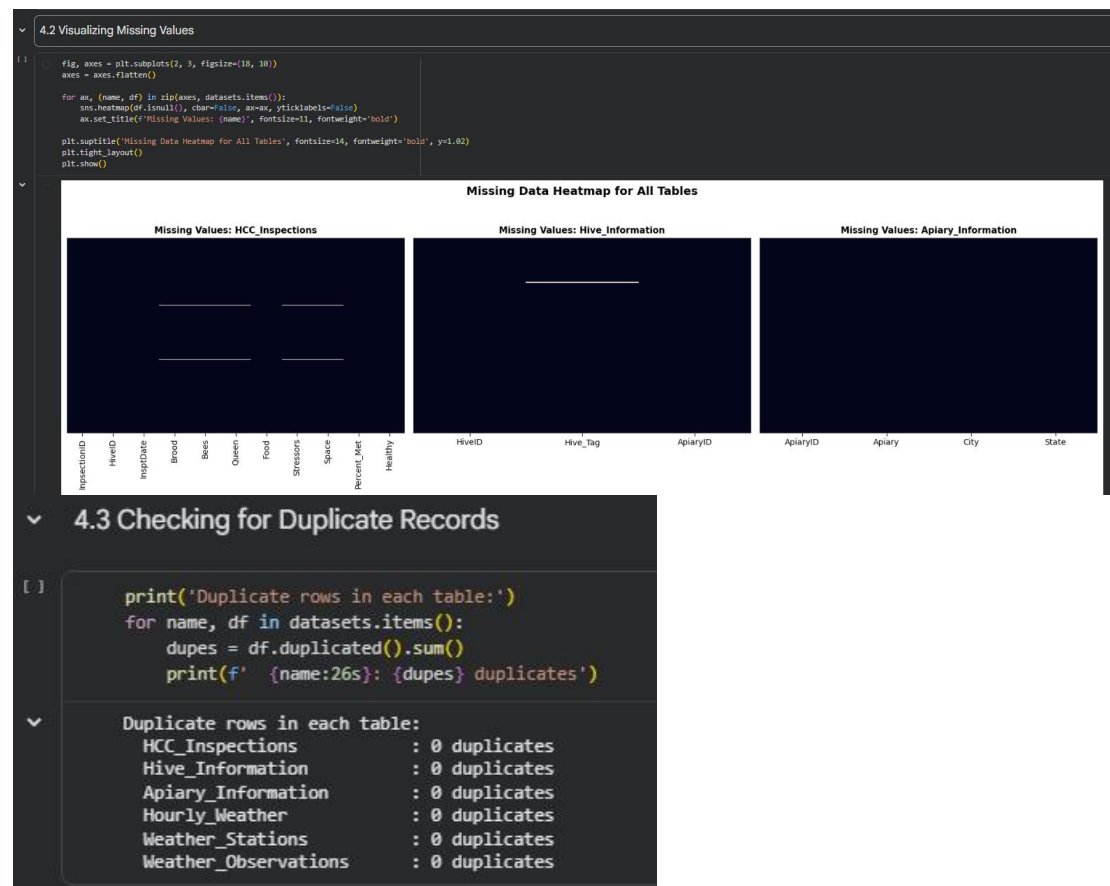
--- Weather_Observations (total missing: 0) ---
No missing values

```

85 missing values in HCC_Inspections.

1 missing value in Hive_Information.

154 missing values in Hourly_Weather.



0 duplicate values found.

5.2 Data Cleaning

Missing values were handled using strategies appropriate to each column type:

- Categorical inspection columns (Brood, Bees, Queen, Food, Space, Stressors) were filled using mode imputation - the most frequently occurring value - to preserve the typical hive condition pattern.
- Numerical weather columns (Temperature, Humidity, Dew_Point, Wind_Speed, Wind_Gust, Pressure, Precip) were filled using mean imputation, which is appropriate for continuous measurements.
- The single missing Hive_Tag entry was filled with the placeholder value "Unknown".

5. Data Cleaning

5.1 Handling Missing Values

Strategy used for Data Cleaning:-

- **Categorical columns** in inspections table are filled using **mode** (which is the most frequent value)
- **Numerical columns** in hourly_weather table are filled using **mean** (which is the average value)
- **Hive_Tag** (has only 1 missing value) is filled with 'Unknown'

```
[ ]
# Categorical columns in inspections – fill with mode
inspections.fillna(inspections.mode().iloc[0], inplace=True)

# Process numerical weather columns in hourly_weather
weather_num_cols = ['Temperature', 'Humidity', 'Dew_Point', 'Wind_Speed',
                    'Wind_Gust', 'Pressure', 'Precip']

for col in weather_num_cols:
    if col in hourly_weather.columns:
        # Convert to string, replace non-numeric characters, then convert to numeric
        hourly_weather[col] = pd.to_numeric(
            hourly_weather[col].astype(str).str.replace(r'^\d-9.-', '', regex=True),
            errors='coerce'
        )
        # Fill NaNs with the mean of the column (if not entirely NaN), else fill with 0
        if hourly_weather[col].isnull().all():
            hourly_weather[col].fillna(0, inplace=True) # Fallback if entire column is NaN
        else:
            hourly_weather[col].fillna(hourly_weather[col].mean(), inplace=True)

# Hive_Tag – fill the 1 missing value with 'Unknown'
hive_info['Hive_Tag'].fillna('Unknown', inplace=True)

print('Missing values handled successfully.')
```

Missing values handled successfully.

Remove Duplicates

```
[17]
✓ Os
# Remove duplicates
inspections.drop_duplicates(inplace=True)

print('Duplicate check completed.')
print(f'inspections rows after de-duplication: {len(inspections)}')
```

Duplicate check completed.
inspections rows after de-duplication: 2404

After cleaning, all six tables showed 0 remaining missing values except Hourly_Weather which retained 42 missing values in non-critical columns. All duplicates were confirmed absent. The inspections table retained 2,404 rows after the duplicate check.

5.3 Verifying Data Cleaning - Missing Values After Cleaning

```
[18]
✓ Os
print('Missing values after cleaning:\n')
for name, df in datasets.items():
    total = df.isnull().sum().sum()
    print(f' {name:26s}: {total} missing values remaining')
```

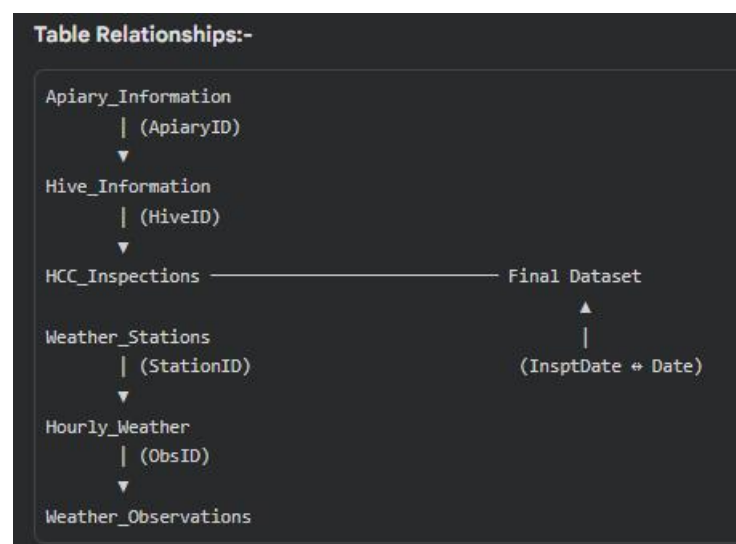
Missing values after cleaning:

HCC_Inspections	: 0 missing values remaining
Hive_Information	: 0 missing values remaining
Apiary_Information	: 0 missing values remaining
Hourly_Weather	: 42 missing values remaining
Weather_Stations	: 0 missing values remaining
Weather_Observations	: 0 missing values remaining

5.3 Data Integration

Since the dataset is distributed across six relational tables, data integration was a required step. The merges were performed in the following order:

1. Hive_Information merged with Apiary_Information on ApiaryID → hive_apiary (188 rows × 6 columns)
2. HCC_Inspections merged with hive_apiary on HiveID → hive_full (2,404 rows × 16 columns)
3. Hourly_Weather merged with Weather_Stations on StationID → weather_station_full
4. weather_station_full merged with Weather_Observations on ObsID → weather_full (3,672 rows × 20 columns)
5. hive_full merged with weather_full on InsptDate = Date → final_dataset (35,656 rows × 36 columns)





The final integrated dataset had 35,656 rows because each inspection matched every hourly weather record from the same day (approximately 15 weather records per inspection). After integration, the missing value check on final_dataset showed 1,720 missing weather values for inspections that had no matching weather date, and 1,874 missing Wind_Direction values.

6.6 Step 5 - Combining Weather with Hive Dataset

In this step, we are integrating weather conditions with hive inspections using the date attribute.

```
[24]
✓ Os
hive_full['InsptDate'] = pd.to_datetime(hive_full['InsptDate'])
weather_full['Date'] = pd.to_datetime(weather_full['Date'])

final_dataset = pd.merge(
    hive_full,
    weather_full,
    left_on='InsptDate',
    right_on='Date',
    how='left'
).copy() # Add .copy() here to ensure final_dataset is a standalone DataFrame

print('Result: Hive Health + Apiary Location + Weather Conditions')
print('This is our final integrated dataset.')
print('Shape:', final_dataset.shape)
print('-' * 40)
print(final_dataset.head())
```

```
... Result: Hive Health + Apiary Location + Weather Conditions
This is our final integrated dataset.
Shape: (35656, 36)
```

```
-----
   InspectionID  HiveID  InsptDate  Brood  Bees  Queen  Food  Stressors  \
0              1       1  2016-06-15    1.0   1.0   1.0   1.0         0.0
1              2       1  2016-07-22    1.0   1.0   1.0   1.0         0.0
2              3       1  2016-08-01    1.0   1.0   1.0   1.0         1.0
3              3       1  2016-08-01    1.0   1.0   1.0   1.0         1.0
4              3       1  2016-08-01    1.0   1.0   1.0   1.0         1.0

   Space  Percent_Met  ...  Precip  Condition  Sunrise  Sunset  \
0      1.0           83  ...    NaN        NaN        NaN    NaN
1      1.0           83  ...    NaN        NaN        NaN    NaN
2      1.0          100  ...    0.0  Light Rain  6:24:00  20:21:00
3      1.0          100  ...    0.0  Light Rain  6:24:00  20:21:00
4      1.0          100  ...    0.0  Light Rain  6:24:00  20:21:00

   Daylight_Hours  Station_City  Station  Date  Obs_Time  Obs_Hour
0              NaN           NaN      NaN  NaT        NaN      NaN
1              NaN           NaN      NaN  NaT        NaN      NaN
2      13:57:00      Clayton    KRDU  2016-08-01    0:51        0.0
3      13:57:00      Clayton    KRDU  2016-08-01    1:51        1.0
4      13:57:00      Clayton    KRDU  2016-08-01    2:51        2.0
```

[5 rows x 36 columns]

Missing values in final_dataset:

```
Wind_Direction    1874
Condition          1800
ObsID             1720
WeatherID         1720
Station           1720
Date              1720
Obs_Time          1720
Obs_Hour          1720
Station_City      1720
Daylight_Hours    1720
StationID         1720
Temperature       1720
Precip            1720
Pressure          1720
Wind_Gust         1720
Wind_Speed        1720
Humidity          1720
Dew_Point         1720
Sunset            1720
Sunrise           1720
Brood              0
InsptDate         0
HiveID            0
InspectionID      0
ApiaryID          0
Apiary            0
City              0
State             0
Stressors         0
Food              0
Queen             0
Bees              0
Space             0
Percent_Met       0
Healthy           0
Hive_Tag          0
dtype: int64
```

7.2 Checking Missing Values

```
[27]
✓ Os print('Missing values (sorted as descending order):')
      print(final_dataset.isnull().sum().sort_values(ascending=False))
```

Missing values (sorted as descending order):

Wind_Direction	1874
Condition	1800
ObsID	1720
WeatherID	1720
Station	1720
Date	1720
Obs_Time	1720
Obs_Hour	1720
Station_City	1720
Daylight_Hours	1720
StationID	1720
Temperature	1720
Precip	1720
Pressure	1720
Wind_Gust	1720
Wind_Speed	1720
Humidity	1720
Dew_Point	1720
Sunset	1720
Sunrise	1720
Brood	0
InsptDate	0
HiveID	0
InspectionID	0
ApiaryID	0
Apiary	0
City	0
State	0
Stressors	0
Food	0
Queen	0
Bees	0
Space	0
Percent_Met	0
Healthy	0
Hive_Tag	0
dtype: int64	

5.4 Post-Integration Data Cleaning

A second round of cleaning was applied after integration. Weather columns stored as object type were converted to numeric. All remaining missing values in numeric columns were filled using mean imputation and categorical columns using mode imputation. Duplicate rows were checked and none were found (final_dataset: 35,656 rows, 0 duplicates). The final_dataset was confirmed fully clean with 0 remaining missing values after this step.


```
Numerical Columns are filled with mean values.

[29]
✓ Os
# Re-process weather numeric columns in final_dataset, in case new NaNs appeared from merging.
weather_features_to_clean_and_fill = ['Temperature', 'Humidity', 'Dew_Point', 'Wind_Speed',
                                       'Wind_Gust', 'Pressure', 'Precip', 'WeatherID', 'ObsID',
                                       'StationID', 'Obs_Hour']

for col in weather_features_to_clean_and_fill:
    if col in final_dataset.columns:
        # Ensure these columns are numeric (coercing errors for cleanliness)
        if final_dataset[col].dtype == 'object':
            final_dataset[col] = final_dataset[col].astype(str).str.replace(r'[^0-9.-]', '', regex=True)
            final_dataset[col] = pd.to_numeric(final_dataset[col], errors='coerce')

# Get all numerical columns that might contain NaNs
numeric_cols_with_nans = final_dataset.select_dtypes(include=['int64', 'float64']).columns

# Calculate means for all numerical columns. For columns that are entirely NaN, their mean will be NaN.
# Then, fill those NaN means with 0, effectively filling entirely NaN columns with 0.
fill_values = final_dataset[numeric_cols_with_nans].mean().fillna(0)

# Apply fillna to the entire set of numerical columns
final_dataset[numeric_cols_with_nans] = final_dataset[numeric_cols_with_nans].fillna(fill_values)

# Handle date column separately
if 'Date' in final_dataset.columns:
    if final_dataset['Date'].dtype == 'object':
        final_dataset['Date'] = pd.to_datetime(final_dataset['Date'], errors='coerce')
        # Fill NaT values for the 'Date' column using ffill/bfill
        final_dataset['Date'] = final_dataset['Date'].ffill().bfill()

print('Missing numerical and date values in final_dataset handled successfully.')

Missing numerical and date values in final_dataset handled successfully.

Categorical Columns are filled with most frequent value (mode).

[30]
✓ Os
categorical_cols = final_dataset.select_dtypes(include=['object']).columns
for col in categorical_cols:
    final_dataset[col].fillna(final_dataset[col].mode()[0], inplace=True)

print('Missing values after handling:', final_dataset.isnull().sum().sum())

Missing values after handling: 0
```

7.5 Removing Duplicate Records

```
[31]
✓ Os
print('Duplicate rows:', final_dataset.duplicated().sum())
final_dataset.drop_duplicates(inplace=True)
print('Rows after dedup:', len(final_dataset))
```

Duplicate rows: 0
Rows after dedup: 35656

```
[32]
✓ Os
print('Final Integrated Dataset Shape:', final_dataset.shape)
print('\nRemaining Missing Values:')
missing_remaining = final_dataset.isnull().sum()
has_missing = missing_remaining[missing_remaining > 0]
print(has_missing if len(has_missing) > 0 else 'None - dataset is clean.')
```

Final Integrated Dataset Shape: (35656, 36)

Remaining Missing Values:
None - dataset is clean.

```
[33]
✓ Os
final_dataset['Date'] = pd.to_datetime(final_dataset['Date'], errors='coerce')

# Verify
print('Date-related column dtypes:')
print(final_dataset[['InsptDate', 'Date']].dtypes)
```

Date-related column dtypes:
InsptDate datetime64[ns]
Date datetime64[ns]
dtype: object

Outliers were detected using the IQR method ($Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$ boundaries). Instead of dropping rows, outlier values were capped at the IQR boundaries to preserve all 35,656 rows. The dataset shape remained $(35,656 \times 36)$ after capping.

7.7 Detecting Outliers Using IQR Method

```
[34]
✓ Os
numeric_only = final_dataset.select_dtypes(include=['float64', 'int64'])

Q1 = numeric_only.quantile(0.25)
Q3 = numeric_only.quantile(0.75)
IQR = Q3 - Q1

# Find outliers
outliers = ((numeric_only < (Q1 - 1.5 * IQR)) |
            (numeric_only > (Q3 + 1.5 * IQR)))

print('Number of outlier values per column (sorted):')
print(outliers.sum().sort_values(ascending=False))
```

Number of outlier values per column (sorted):

Space	6666
Bees	6207
Food	5633
Wind_Gust	5239
Pressure	2702
Precip	2108
Percent_Met	2088
ApiaryID	1220
Temperature	482
Wind_Speed	426
Dew_Point	218
HiveID	0
InspectionID	0
Queen	0
Brood	0
WeatherID	0
Stressors	0
StationID	0
ObsID	0
Humidity	0
Obs_Hour	0
dtype: int64	


```
7.8 Handle Outliers - Capping Extreme Values

We have capped or clipped outlier values at the IQR boundaries instead of removing entire rows, to preserve data volume.

[35] ✓ Os
for col in numeric_only.columns:
    lower = Q1[col] - 1.5 * IQR[col]
    upper = Q3[col] + 1.5 * IQR[col]
    final_dataset[col] = final_dataset[col].clip(lower=lower, upper=upper)

print('Outliers capped. Dataset shape:', final_dataset.shape)

Outliers capped. Dataset shape: (35656, 36)

7.10 Final Data Quality Check

[37] ✓ Os
print('Final Integrated Dataset Shape:', final_dataset.shape)
print('\nRemaining Missing Values:')
missing_remaining = final_dataset.isnull().sum()
has_missing = missing_remaining[missing_remaining > 0]
print(has_missing if len(has_missing) > 0 else 'None - dataset is clean.')

Final Integrated Dataset Shape: (35656, 36)

Remaining Missing Values:
None - dataset is clean.
```

5.5 Noise Reduction - Binning and Smoothing

Weather sensor readings contain natural short-term fluctuations. Four techniques were applied:

```
8. Noise Reduction Using Binning and Smoothing

Noise reduction using binning and smoothing is a classic technique used in Data Mining. It is important especially because weather sensor readings often contain short term fluctuations and random measurement noise. Adding noise reduction has made our preprocessing aligned with Data Mining steps.

Methods used are:

1. Equal Width Binning
2. Mean Smoothing within bins
3. Rolling Average (Time Series Smoothing)
4. Regression Smoothing
```

- **Equal-width binning:** Temperature was divided into 4 labeled intervals - Cold, Moderate, Warm, and Hot - using `pd.cut()` with `bins=4`.

8.1 Equal Width Binning (Temperature)

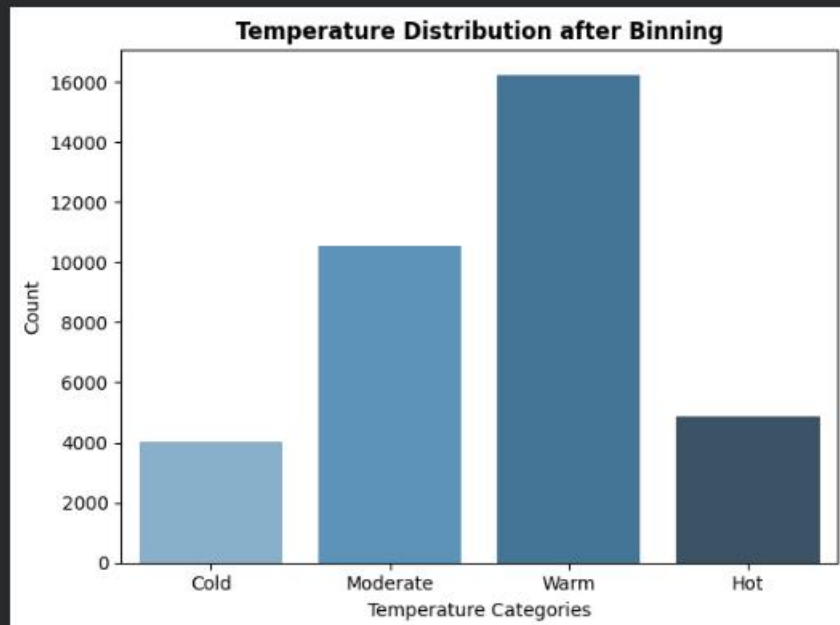
Binning groups continuous values into intervals to reduce noise. This step reduces small fluctuations in temperature readings.

```
[38] # Create bins for temperature
✓ Os final_dataset['Temp_Bin'] = pd.cut(
    final_dataset['Temperature'],
    bins=4,
    labels=['Cold', 'Moderate', 'Warm', 'Hot']
)

print('Temperature Bins sample:')
print(final_dataset[['Temperature', 'Temp_Bin']].head(10))
```

```
Temperature Bins sample:
  Temperature Temp_Bin
0  70.507019      Warm
1  70.507019      Warm
2  73.000000      Warm
3  74.000000      Warm
4  74.000000      Warm
5  73.000000      Warm
6  73.000000      Warm
7  73.000000      Warm
8  74.000000      Warm
9  75.000000      Warm
```

```
[40] sns.countplot(data=final_dataset, x='Temp_Bin', palette='Blues_d')
✓ Os plt.title('Temperature Distribution after Binning', fontweight='bold')
plt.xlabel('Temperature Categories')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```



- **Equal-frequency binning:** Temperature was also divided into 5 quantile-based bins using `pd.qcut()` with `q=5`.
- **Mean smoothing within bins:** Each temperature value was replaced with the mean of its bin, producing a smoothed column (`Temp_Smoothed`).

8.3 Mean Smoothing Within Bins

Here we are replacing values inside each bin with the mean of the bin. This will help to remove random noise.

```
[41]
✓ Os
# Equal Width Binning
final_dataset['Temp_Bins'] = pd.cut(final_dataset['Temperature'], bins=5)
print('Equal Width Bins sample:')
print(final_dataset[['Temperature', 'Temp_Bins']].head())

# Equal Frequency Binning - better when data is not evenly distributed
final_dataset['Temp_Freq_Bins'] = pd.qcut(
    final_dataset['Temperature'], q=5, duplicates='drop'
)
print('\nEqual Frequency Bins sample:')
print(final_dataset[['Temperature', 'Temp_Freq_Bins']].head())

# Mean Smoothing within bins
bins_cut = pd.cut(final_dataset['Temperature'], bins=5)
final_dataset['Temp_Smoothed'] = final_dataset.groupby(
    bins_cut, observed=False
)['Temperature'].transform('mean')
print('\nSmoothed Temperature sample:')
print(final_dataset[['Temperature', 'Temp_Smoothed']].head())
```

Equal Width Bins sample:

	Temperature	Temp_Bins
0	70.507019	(64.2, 74.8]
1	70.507019	(64.2, 74.8]
2	73.000000	(64.2, 74.8]
3	74.000000	(64.2, 74.8]
4	74.000000	(64.2, 74.8]

Equal Frequency Bins sample:

	Temperature	Temp_Freq_Bins
0	70.507019	(69.0, 73.0]
1	70.507019	(69.0, 73.0]
2	73.000000	(69.0, 73.0]
3	74.000000	(73.0, 79.0]
4	74.000000	(73.0, 79.0]

Smoothed Temperature sample:

	Temperature	Temp_Smoothed
0	70.507019	69.904328
1	70.507019	69.904328
2	73.000000	69.904328
3	74.000000	69.904328
4	74.000000	69.904328

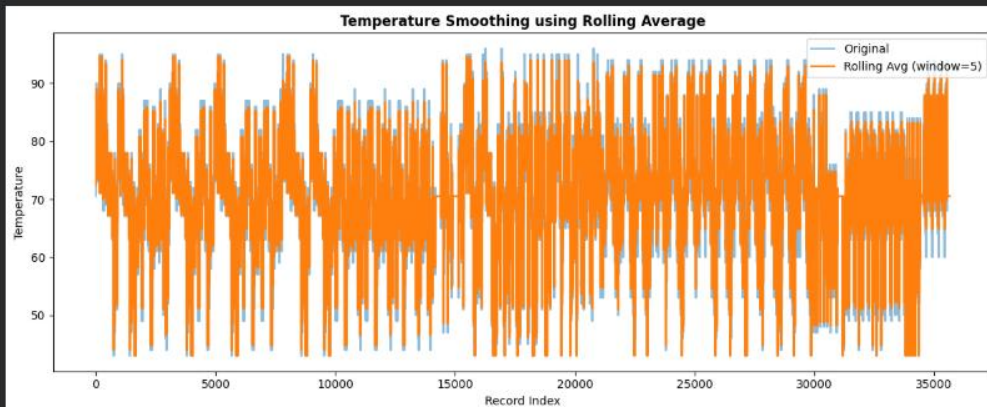
- **Rolling average smoothing:** A 5-record moving average was applied to the temperature series (Temp_RollingAvg).

8.5 Rolling Average (Time Series Smoothing)

Weather data often contains short term fluctuations. A rolling average smoothens the trend over a window of consecutive readings.

```
[43] 15 final_dataset['Temp_RollingAvg'] = final_dataset['Temperature'].rolling(window=5).mean()

plt.figure(figsize=(12, 5))
plt.plot(final_dataset['Temperature'].values, label='Original', alpha=0.5)
plt.plot(final_dataset['Temp_RollingAvg'].values, label='Rolling Avg (window=5)', linewidth=1.5)
plt.legend()
plt.title('Temperature Smoothing using Rolling Average', fontweight='bold')
plt.xlabel('Record Index')
plt.ylabel('Temperature')
plt.tight_layout()
plt.show()
```

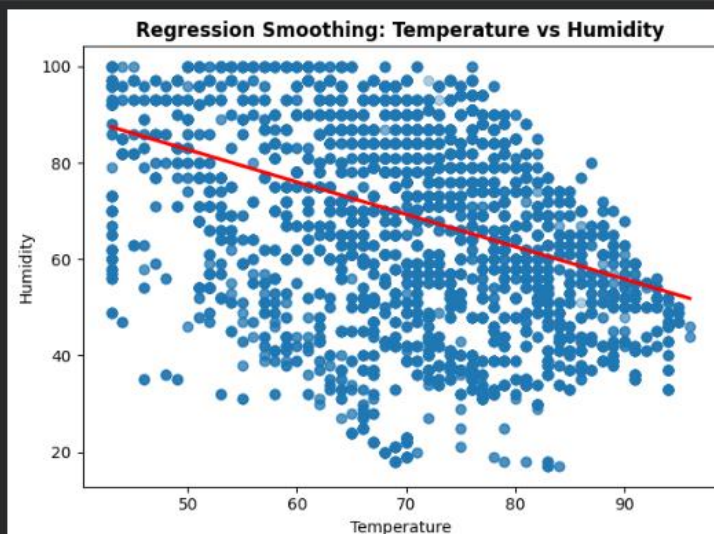


- **Regression smoothing:** A regression line was fitted between Temperature and Humidity to visualize their smoothed relationship.

8.6 Regression Smoothing

We could fit a regression line to smooth noisy relationships between two variables. This has showed a smoothed relationship between Temperature and Humidity.

```
[44] 15 sns.regplot(
    x=final_dataset['Temperature'],
    y=final_dataset['Humidity'],
    scatter_kws={'alpha': 0.2},
    line_kws={'color': 'red'}
)
plt.title('Regression Smoothing: Temperature vs Humidity', fontweight='bold')
plt.tight_layout()
plt.show()
```



Mean smoothing was then applied to all four key weather features - Temperature, Humidity, Wind_Speed, and Pressure - producing smoothed columns: Temperature_Smoothed, Humidity_Smoothed, Wind_Speed_Smoothed, and Pressure_Smoothed (confirmed from notebook output).

```

8.7 Applying Mean Smoothing to All Key Weather Features

[45]
✓ Os
weather_features = ['Temperature', 'Humidity', 'Wind_Speed', 'Pressure']

for feature in weather_features:
    if feature in final_dataset.columns:
        bins_cut = pd.cut(final_dataset[feature], bins=5)
        final_dataset[f'{feature}_Smoothed'] = final_dataset.groupby(
            bins_cut, observed=False
        )[feature].transform('mean')

print('Smoothed columns added:', [f'{f}_Smoothed' for f in weather_features])

Smoothed columns added: ['Temperature_Smoothed', 'Humidity_Smoothed', 'Wind_Speed_Smoothed', 'Pressure_Smoothed']

```

5.6 Feature Aggregation

To resolve the one-to-many row explosion from the hourly weather join, hourly weather records were aggregated to daily summaries. For each day, the following statistics were computed: mean, minimum, and maximum temperature; mean, minimum, and maximum humidity; mean and maximum wind speed; mean pressure; and total precipitation. These were stored as Temp_Mean, Temp_Min, Temp_Max, Humidity_Mean, Humidity_Min, Humidity_Max, WindSpeed_Mean, WindSpeed_Max, Pressure_Mean, and Precip_Total. After merging these daily aggregates back into final_dataset, the shape became (35,656 × 56). A sample from the notebook shows Temp_Mean = 80.21°F and Humidity_Mean = 80.375% for inspections on 2016-08-01.

```

9. Feature Aggregation

Feature Aggregation = combining hourly weather into a form that matches inspection-level records.
Feature Engineering = creating new meaningful variables like Hive Stress Index and Weather Stress Index.

Our hive inspections are periodic but weather data is hourly. So instead of attaching raw hourly weather rows directly,
aggregating hourly weather into daily summaries that match each hive inspection date.

9.1 Step A - Converting Date Columns

[46]
✓ Os
# Convert to datetime
final_dataset['Date'] = pd.to_datetime(final_dataset['Date'], errors='coerce')

# Before converting hourly_weather['Date'], first ensure 'Date' column exists in hourly_weather
# by merging it with weather_observations. Assuming 'ObsID' is the common column to merge.
hourly_weather = pd.merge(hourly_weather, weather_observations[['ObsID', 'Date']], on='ObsID', how='left')

# Now convert the 'Date' column in hourly_weather
hourly_weather['Date'] = pd.to_datetime(hourly_weather['Date'], errors='coerce')

9.2 Step B - Creating Daily Weather Aggregates

This step will convert hourly weather into daily weather features.

[47]
✓ Os
# Convert weather columns to numeric first
for col in ['Temperature', 'Humidity', 'Wind_Speed', 'Pressure', 'Precip']:
    if col in hourly_weather.columns:
        hourly_weather[col] = pd.to_numeric(hourly_weather[col], errors='coerce')

# Create a daily date column
hourly_weather['Day'] = hourly_weather['Date'].dt.date

# Aggregate hourly weather into daily summaries
daily_weather = hourly_weather.groupby('Day').agg({
    'Temperature': ['mean', 'min', 'max'],
    'Humidity': ['mean', 'min', 'max'],
    'Wind_Speed': ['mean', 'max'],
    'Pressure': ['mean'],
    'Precip': ['sum']
}).reset_index()

```

9.3 Step C - Flatten Column Names

```
[48]
✓ Os
# Flatten multi-level column names
daily_weather.columns = [
    'Day',
    'Temp_Mean', 'Temp_Min', 'Temp_Max',
    'Humidity_Mean', 'Humidity_Min', 'Humidity_Max',
    'WindSpeed_Mean', 'WindSpeed_Max',
    'Pressure_Mean',
    'Precip_Total'
]

print('Daily weather aggregates:')
print(daily_weather.head())
```

Daily weather aggregates:

	Day	Temp_Mean	Temp_Min	Temp_Max	Humidity_Mean	Humidity_Min	\
0	2016-03-29	55.166667	45.0	66.0	47.458333	29.0	
1	2016-04-21	66.500000	53.0	81.0	60.791667	44.0	
2	2016-04-28	70.625000	66.0	83.0	81.750000	56.0	
3	2016-05-02	72.000000	66.0	83.0	80.916667	56.0	
4	2016-05-09	70.166667	59.0	80.0	73.583333	52.0	

	Humidity_Max	WindSpeed_Mean	WindSpeed_Max	Pressure_Mean	Precip_Total
0	71.0	3.583333	12.0	29.653333	0.0
1	78.0	9.083333	14.0	29.679167	0.0
2	96.0	5.500000	13.0	29.417917	1.2
3	93.0	8.958333	22.0	29.495417	0.2
4	93.0	1.458333	6.0	29.593750	0.2

9.4 Step D - Match Daily Weather with Inspections

Now we have aligned daily weather to the inspection date.

```
[49]
✓ Os
# Create inspection day column
final_dataset['Day'] = final_dataset['Date'].dt.date

# Merge aggregated weather into final_dataset
final_dataset = pd.merge(
    final_dataset,
    daily_weather,
    on='Day',
    how='left'
)

print('Shape after weather aggregation merge:', final_dataset.shape)
print(final_dataset[['Day', 'Temp_Mean', 'Humidity_Mean', 'WindSpeed_Mean']].head())
```

Shape after weather aggregation merge: (35656, 56)

	Day	Temp_Mean	Humidity_Mean	WindSpeed_Mean
0	2016-08-01	80.208333	80.375	5.75
1	2016-08-01	80.208333	80.375	5.75
2	2016-08-01	80.208333	80.375	5.75
3	2016-08-01	80.208333	80.375	5.75
4	2016-08-01	80.208333	80.375	5.75

5.7 Feature Engineering

Four new features were constructed to capture biologically and environmentally meaningful information:

- **Hive Stress Index:** Combines the five binary inspection attributes (Queen, Bees, Brood, Food, Space) into a single composite score by averaging them. A higher value indicates a more stressed colony.

10. Feature Engineering

Feature Engineering has created new, biologically meaningful predictors from existing data.

We will be creating:-

- **Hive Stress Index** - it is a custom composite score from hive inspection conditions.
- **Weather Stress Index** - it quantifies environmental stress on colonies.
- **Combined Colony Risk Score** - it merges both indices into an overall risk indicator.

10.1 Hive Stress Index

Higher `Hive_Stress_Index` means more stressed hive whereas Lower `Hive_Stress_Index` means healthier hive. If hive conditions are text categories, we have to map them first. If they are already numeric, we can simply skip the mapping.

```
[50]
✓ Os
# Mapping if values are text categories (skip if already numeric)
condition_map = {'Poor': 2, 'Average': 1, 'Good': 0}

for col in ['Queen', 'Bees', 'Brood', 'Food', 'Space']:
    if final_dataset[col].dtype == 'object':
        final_dataset[col] = final_dataset[col].map(condition_map)

# Create Hive Stress Index
final_dataset['Hive_Stress_Index'] = (
    final_dataset['Queen'].fillna(0) +
    final_dataset['Bees'].fillna(0) +
    final_dataset['Brood'].fillna(0) +
    final_dataset['Food'].fillna(0) +
    final_dataset['Space'].fillna(0)
) / 5

print('Hive Stress Index statistics:')
print(final_dataset['Hive_Stress_Index'].describe())
```

```

Hive Stress Index statistics:
count    35656.000000
mean      0.894828
std       0.154634
min       0.600000
25%       0.800000
50%       1.000000
75%       1.000000
max       1.000000
Name: Hive_Stress_Index, dtype: float64

```

- **Weather Stress Index:** Uses threshold-based rules - temperature outside 15°C to 30°C, humidity outside 40% to 70%, wind speed above 15, and any precipitation - each flagged as 1, then averaged across four stress components.

10.2 Weather Stress Index

This step captures whether weather conditions are unfavorable for bees. Stress components will include:

- **Temperature** = outside ideal bee activity range (~15°C to 30°C)
- **Humidity** = outside moderate range (~40% to 70%)
- **Wind speed** > 15 (strong winds can reduce bee flight)
- **Precipitation** > 0 (rainfall reduces foraging)

```
[51]
✓ Os
# Temperature stress
final_dataset['Temp_Stress'] = np.where(
    (final_dataset['Temp_Mean'] < 15) | (final_dataset['Temp_Mean'] > 30), 1, 0
)

# Humidity stress
final_dataset['Humidity_Stress'] = np.where(
    (final_dataset['Humidity_Mean'] < 40) | (final_dataset['Humidity_Mean'] > 70), 1, 0
)

# Wind stress
final_dataset['Wind_Stress'] = np.where(
    final_dataset['WindSpeed_Mean'] > 15, 1, 0
)

# Rain stress
final_dataset['Rain_Stress'] = np.where(
    final_dataset['Precip_Total'] > 0, 1, 0
)
```

- **Colony Risk Score:** A weighted combination giving 60% weight to Hive Stress Index and 40% to Weather Stress Index.

10.4 Combined Colony Risk Score

Hive inspection condition is given a little higher importance (60%) than weather (40%) because it directly relates to inspection time.

```
[53]
✓ Os
# Combined colony risk score
final_dataset['Colony_Risk_Score'] = (
    0.6 * final_dataset['Hive_Stress_Index'] +
    0.4 * final_dataset['Weather_Stress_Index']
)

print('Sample of engineered features:')
print(final_dataset[['Hive_Stress_Index', 'Weather_Stress_Index', 'Colony_Risk_Score']].head())
```

```
Sample of engineered features:
   Hive_Stress_Index  Weather_Stress_Index  Colony_Risk_Score
0                  1.0                   0.5                0.8
1                  1.0                   0.5                0.8
2                  1.0                   0.5                0.8
3                  1.0                   0.5                0.8
4                  1.0                   0.5                0.8
```

- **Temperature Range:** The daily difference between maximum and minimum temperature (Temp_Max – Temp_Min).

```

10.5 Temperature Range Feature

[54] ✓ Os
# Temperature Range Feature
final_dataset['Temp_Range'] = final_dataset['Temp_Max'] - final_dataset['Temp_Min']

print("Temperature Range feature added.")
print(final_dataset['Temp_Range'].describe())

Temperature Range feature added.
count      35656.000000
mean         21.156457
std           5.720249
min          11.000000
25%          17.000000
50%          21.000000
75%          25.000000
max          34.000000
Name: Temp_Range, dtype: float64

```

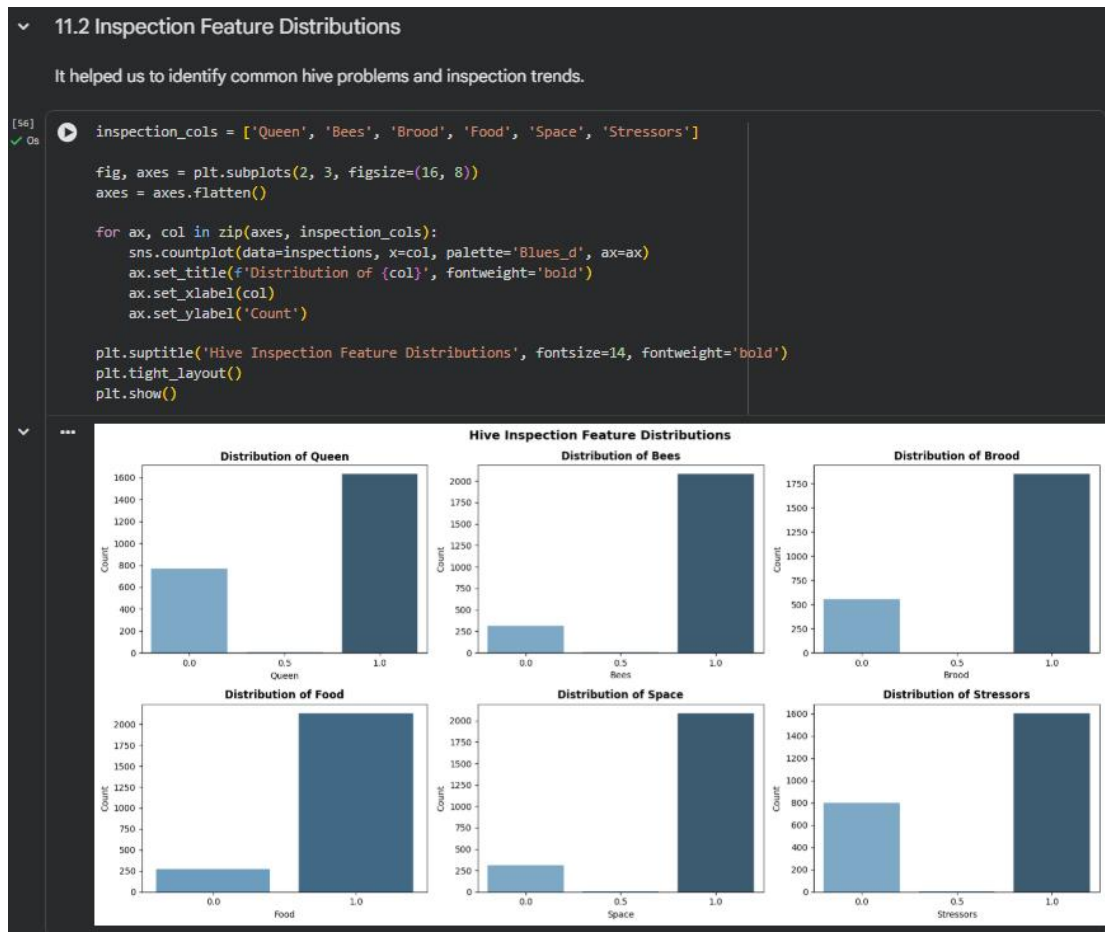
5.8 Exploratory Data Analysis

A total of fifteen visualizations were produced during EDA:

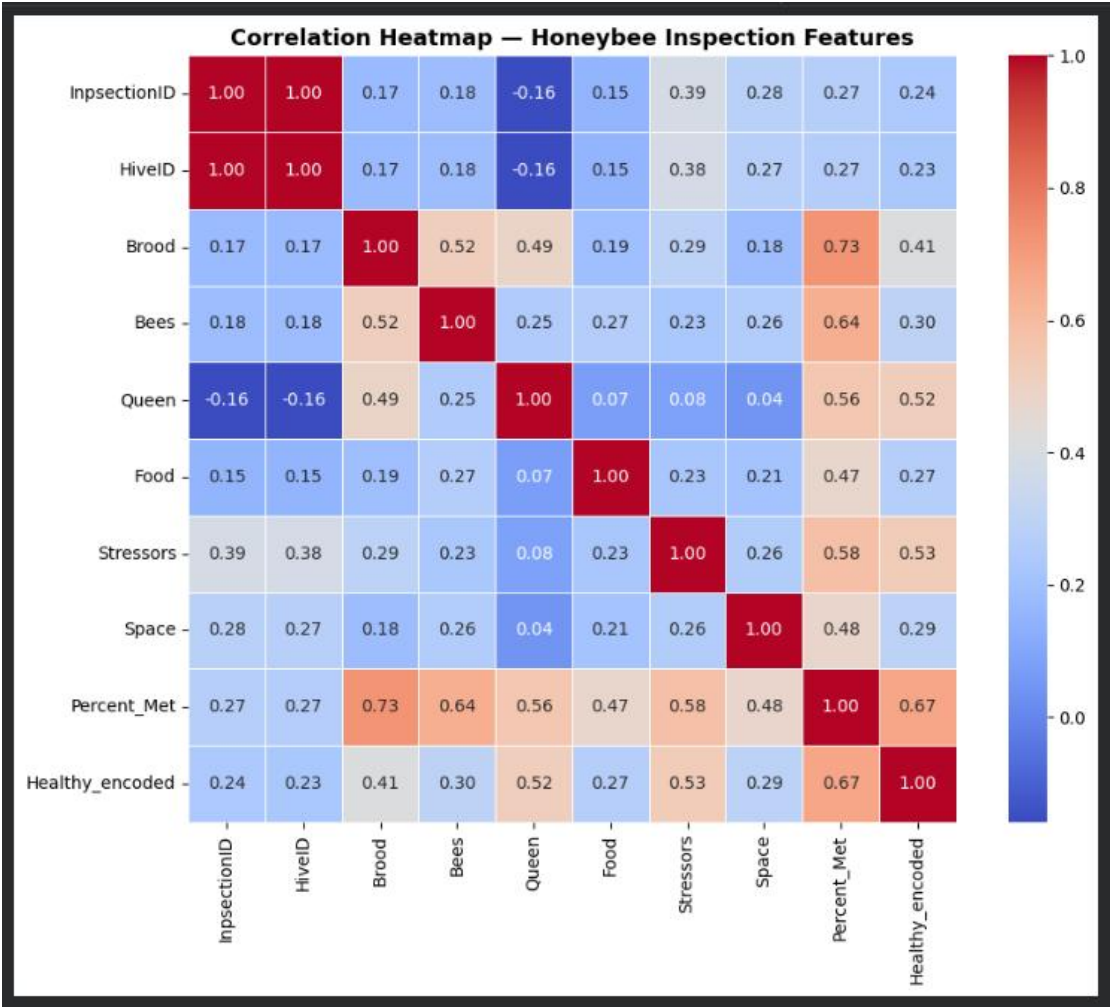
- **Hive health distribution:** 63.7% unhealthy vs 36.3% healthy - class imbalance confirmed.



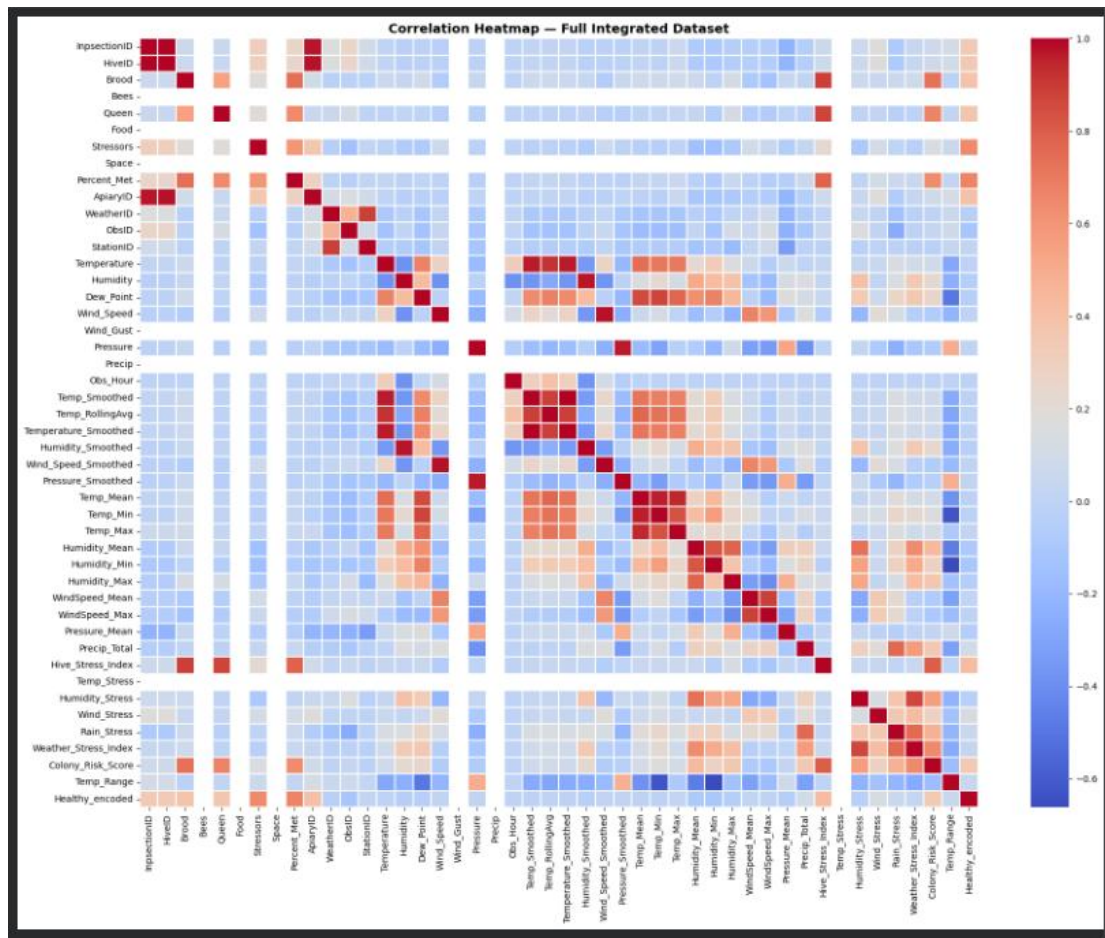
- **Inspection feature distributions:** bar charts for Queen, Bees, Brood, Food, Space, and Stressors.



- **Correlation heatmap** (inspection level): shows relationships between binary inspection attributes and Healthy_encoded.



- **Correlation heatmap (full integrated dataset):** shows all numeric feature correlations including weather variables.

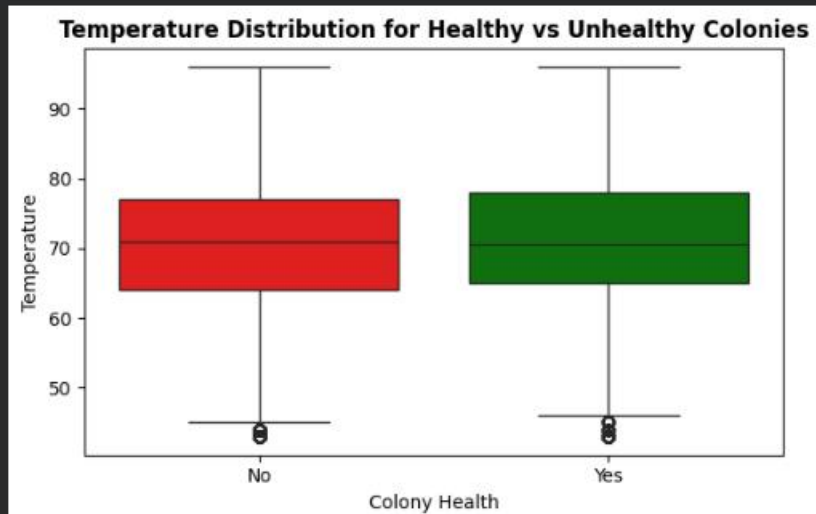


- **Temperature vs colony health (boxplot):** examines whether temperature distributions differ between healthy and unhealthy colonies.

11.5 Weather Influence Analysis - Temperature vs Hive Health

This will check if temperature affects colony health.

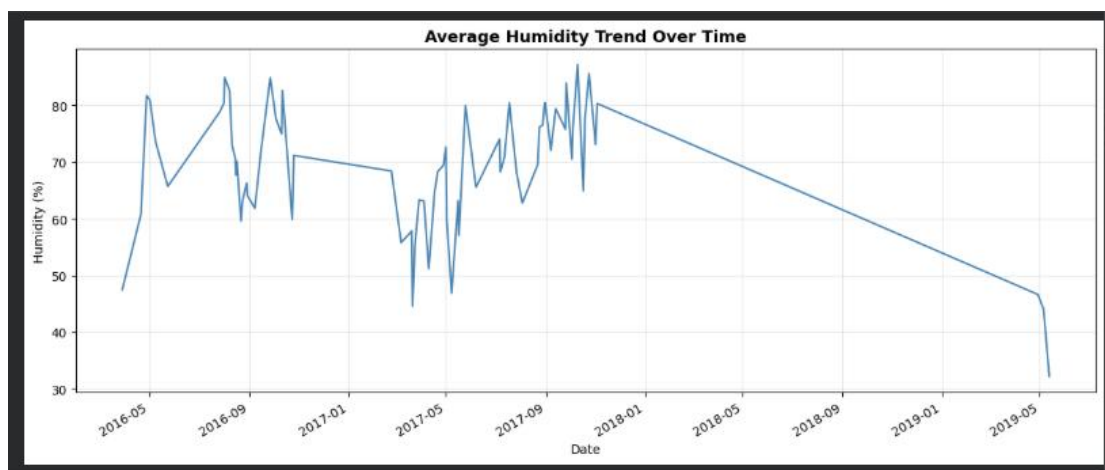
```
[59]  
✓ 0s plt.figure(figsize=(6, 4))  
sns.boxplot(  
    x='Healthy',  
    y='Temperature',  
    data=final_dataset,  
    palette={'Yes': 'green', 'No': 'red'}  
)  
plt.title('Temperature Distribution for Healthy vs Unhealthy Colonies', fontweight='bold')  
plt.xlabel('Colony Health')  
plt.ylabel('Temperature')  
plt.tight_layout()  
plt.show()
```



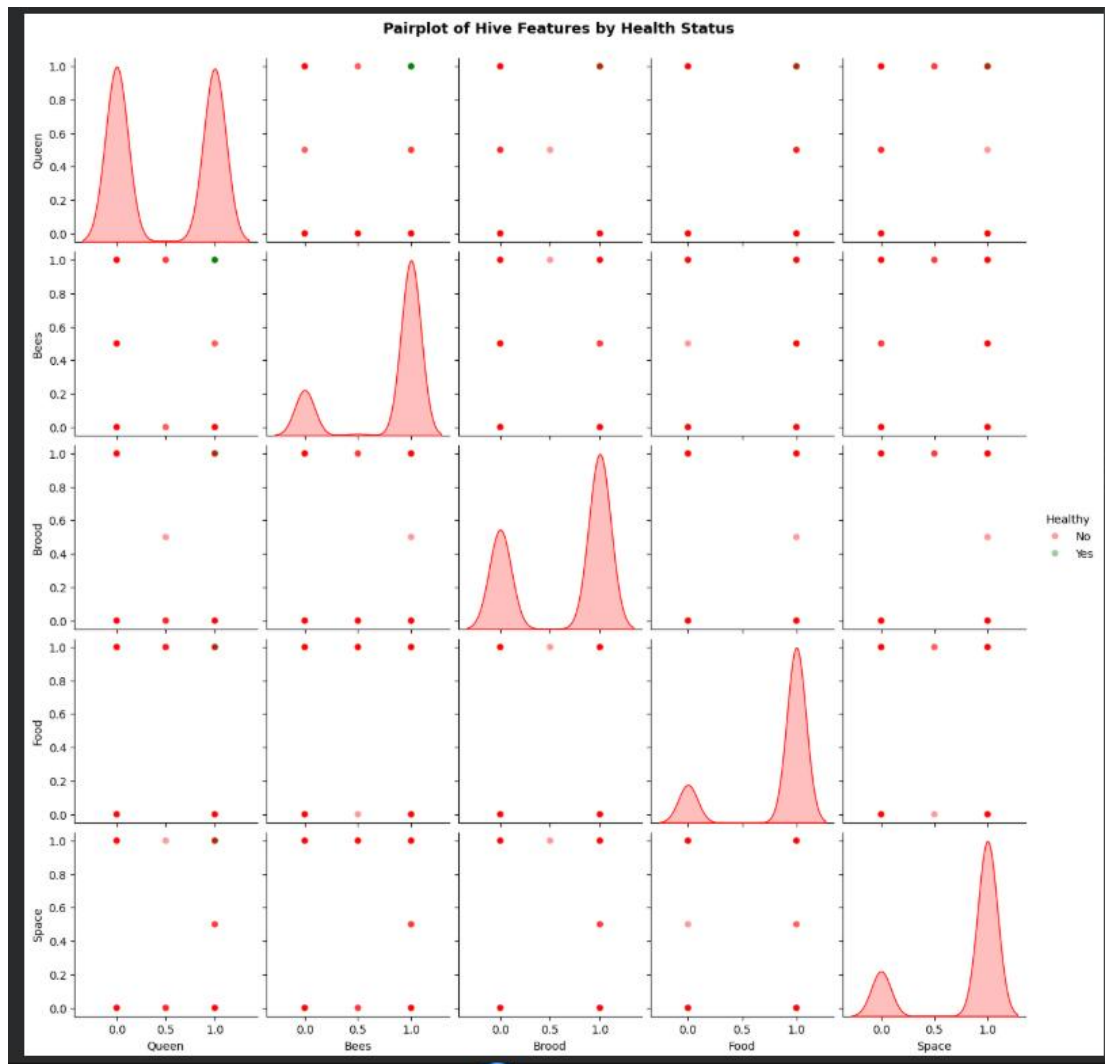
- **Hive health trend over time:** line chart of average health score by inspection date from 2016 to 2019.



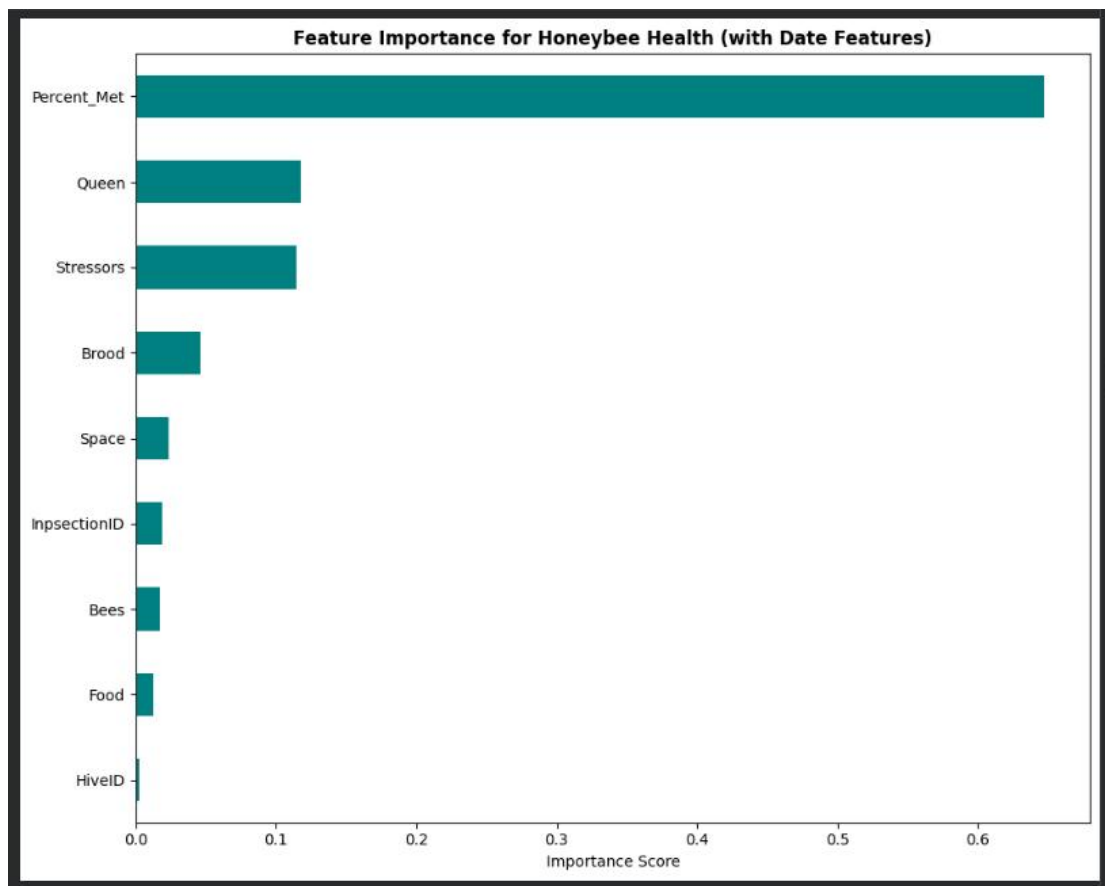
- **Average humidity trend over time:** time series of daily humidity across the dataset period.



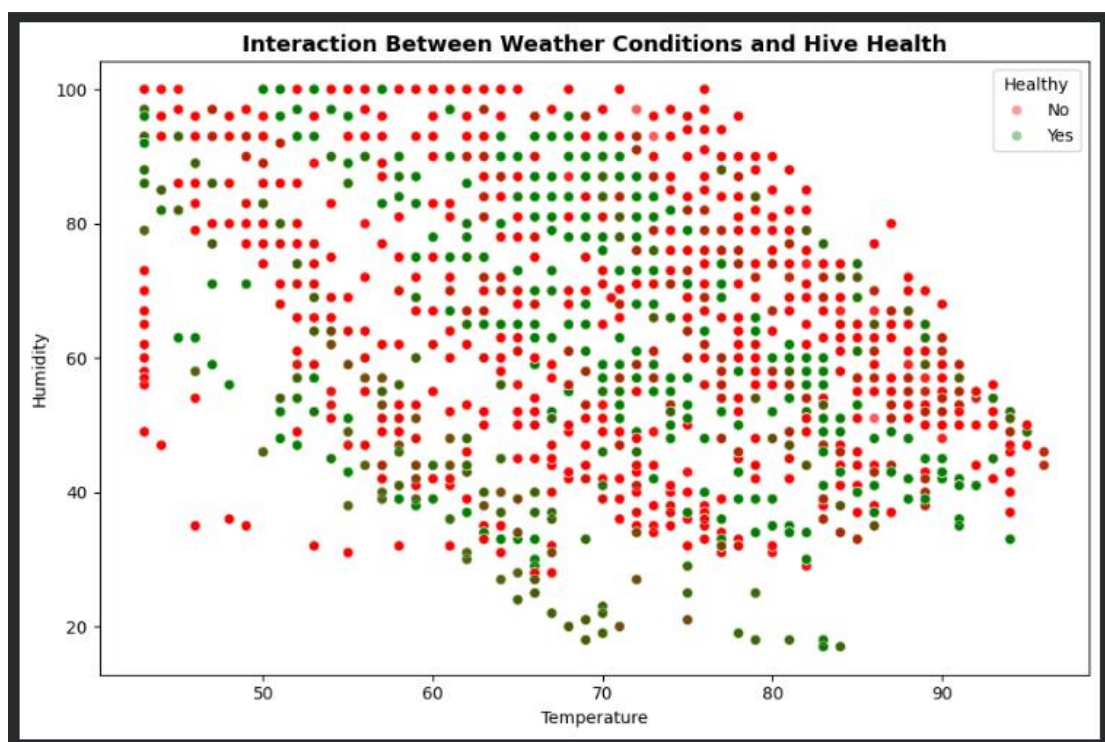
- **Pairplot of inspection features:** scatter matrix colored by health status showing inter-feature relationships.



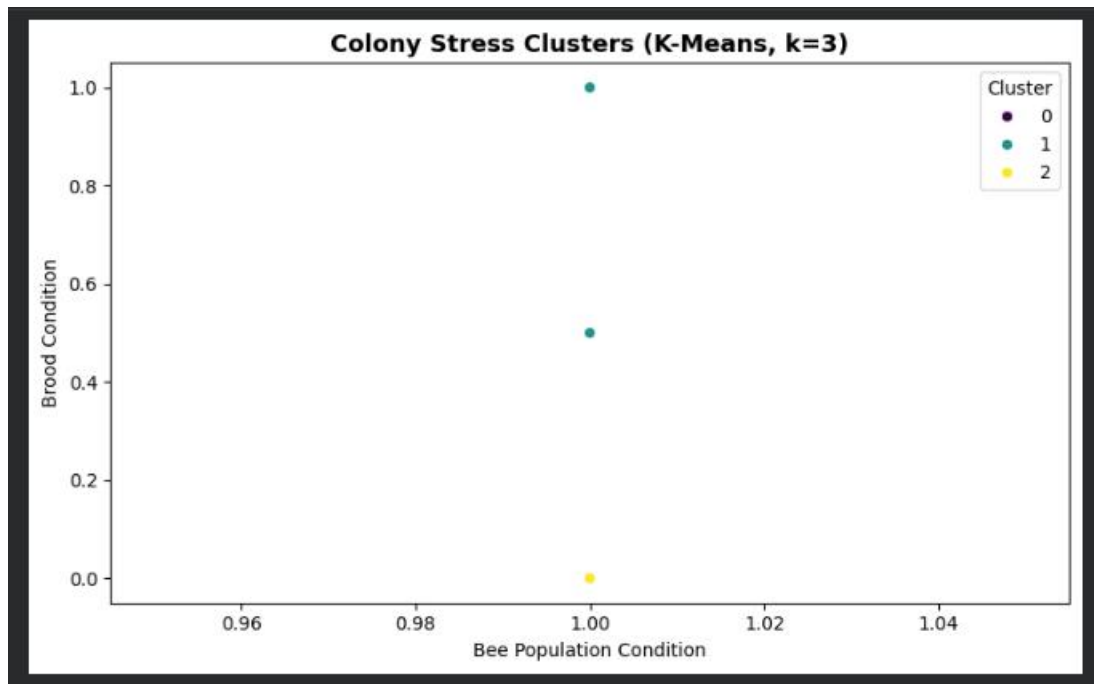
- **Feature importance (pre-ML):** Random Forest is applied to the inspection table, with Percent_Met and Stressors ranking highest before leakage was removed.



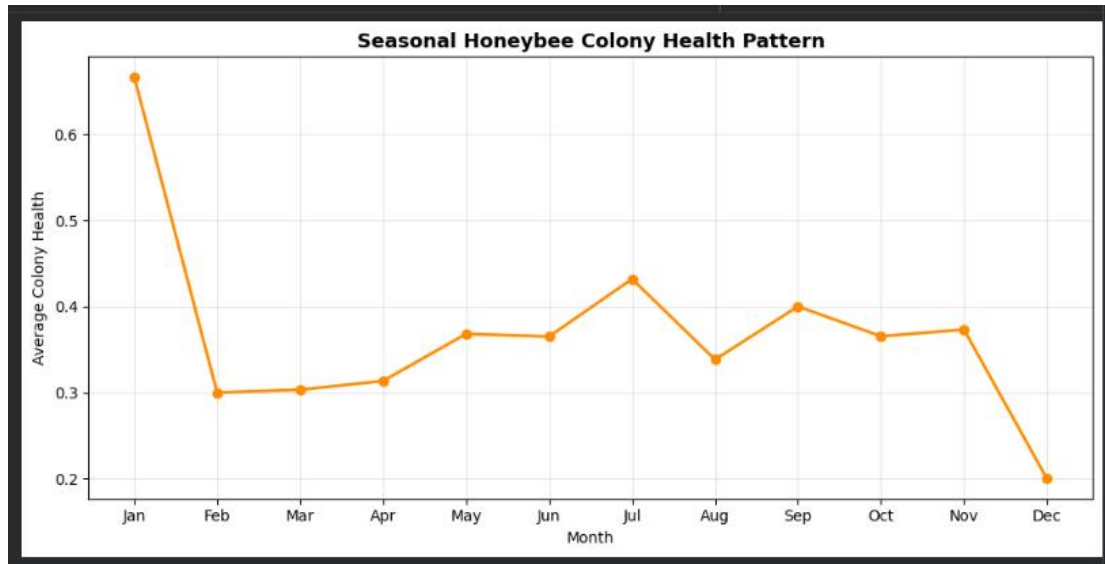
- **Hive health vs weather interaction plot:** temperature vs humidity scatter colored by Healthy status.



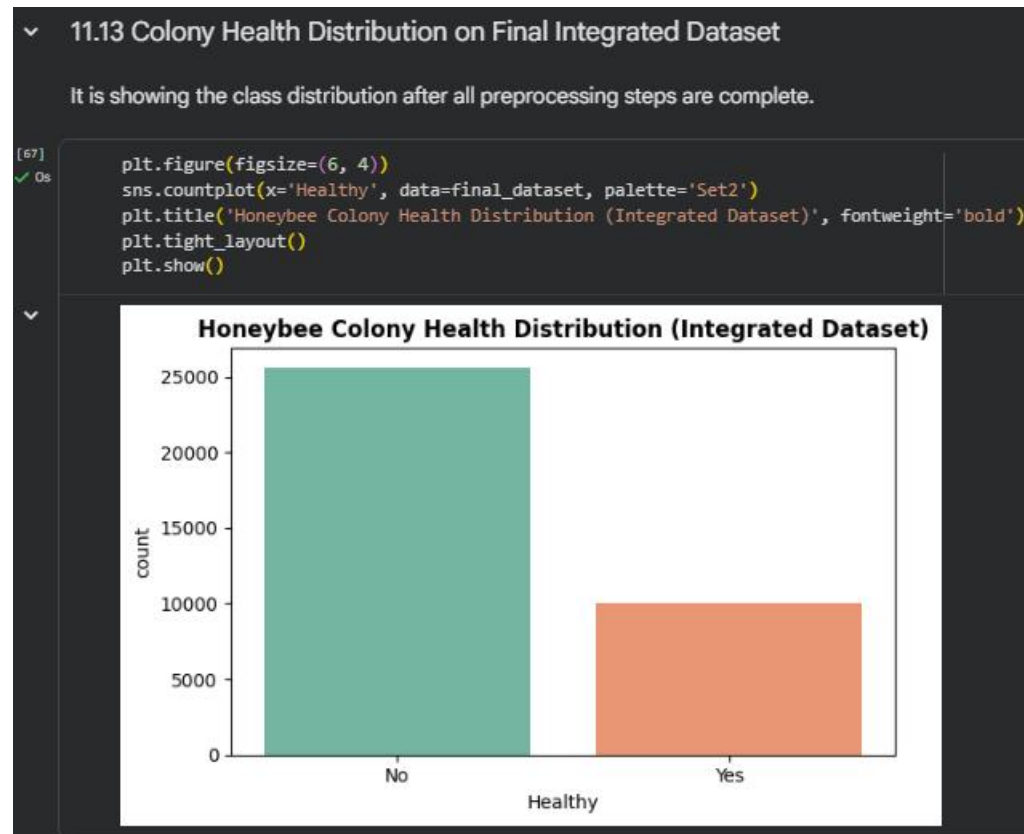
- **Colony stress clusters:** K-Means (k=3) on Bees and Brood features identifying three colony condition groups.



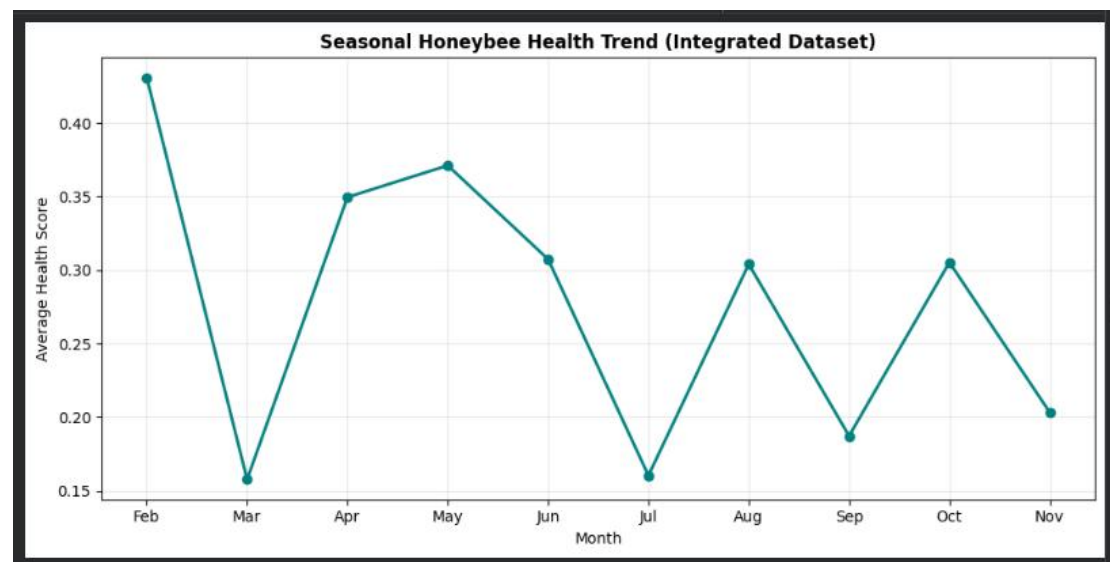
- **Seasonal analysis (inspections):** monthly average health score showing variation across the year.



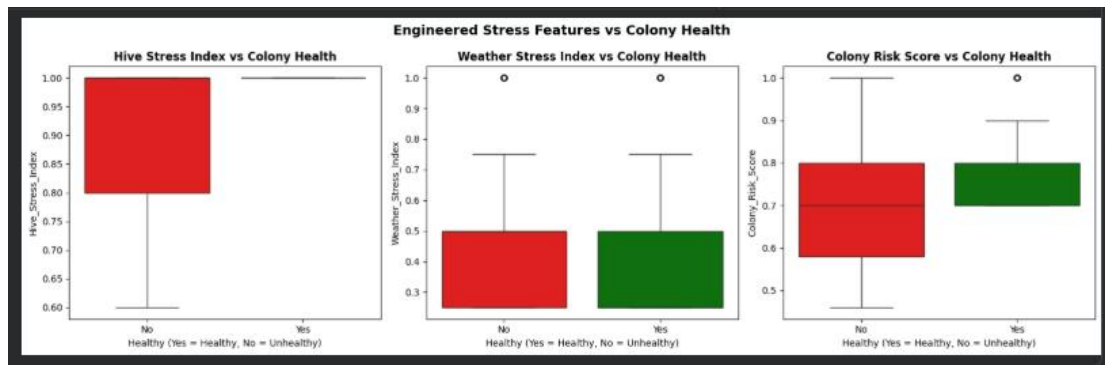
- **Colony health distribution on final integrated dataset:** countplot after full preprocessing.



- **Seasonal health trend on integrated dataset:** monthly groupby on Healthy_encoded.



- **Engineered features vs colony health:** three side-by-side boxplots for Hive Stress Index, Weather Stress Index, and Colony Risk Score separated by Healthy status.



5.9 Feature Encoding and Normalization

Categorical columns were encoded using Label Encoding. Columns excluded from encoding included date-related and bin columns (InsptDate, Date, Obs_Time, Sunrise, Sunset, Daylight_Hours, Temp_Bin, Temp_Bins, Temp_Freq_Bins). After encoding, the remaining object columns were Sunrise, Sunset, Daylight_Hours, and Obs_Time (confirmed from notebook output). Min-Max Scaling was applied to all numeric columns, bringing all values into the range [0, 1].

12. Feature Encoding

Now, we are going to encode the categorical variables. Categorical variables have to be converted to numerical form before applying machine learning algorithms.

```
[70]
✓ Os
# FEATURE ENCODING
label = LabelEncoder()

# Encode all object columns (excluding date and bin columns)
skip_cols = ['InsptDate', 'Date', 'Obs_Time', 'Sunrise', 'Sunset',
             'Daylight_Hours', 'Temp_Bin', 'Temp_Bins', 'Temp_Freq_Bins']

for col in final_dataset.select_dtypes(include='object').columns:
    if col not in skip_cols:
        try:
            final_dataset[col] = label.fit_transform(final_dataset[col].astype(str))
        except Exception as e:
            print(f'Skipped encoding {col}: {e}')

print('Encoding complete. Remaining object columns:')
print(final_dataset.select_dtypes(include='object').columns.tolist())

Encoding complete. Remaining object columns:
['Sunrise', 'Sunset', 'Daylight_Hours', 'Obs_Time']
```


13. Data Normalization

Now, we are scaling numerical features. Weather attributes have different scales like Temperature is -10 to 40, Humidity is 0 to 100, Pressure is 950 to 1050. Min-Max Scaling normalizes all values to the range [0, 1].

Formula

$$X' = (X - X_{\min}) / (X_{\max} - X_{\min})$$

```
[71]
✓ Os
# NORMALIZATION
scaler = MinMaxScaler()

# Scale only numeric columns; skip date columns
num_cols = final_dataset.select_dtypes(include=['int64', 'float64', 'int32']).columns
final_dataset[num_cols] = scaler.fit_transform(final_dataset[num_cols])

print('Normalization complete.')
print('Feature ranges after scaling (min / max):')
print(final_dataset[num_cols].describe().loc[['min', 'max']].to_string())
```

Normalization complete.
Feature ranges after scaling (min / max):

	InpsectionID	HiveID	Brood	Bees	Queen	Food	Stressors	Space	Percent_Met	Healthy	Hive_Tag	ApiaryID	Apiary	Cit
min	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
max	1.0	1.0	1.0	0.0	1.0	0.0	1.0	0.0	1.0	1.0	1.0	1.0	1.0	1.0

15. Extracting Independent and Dependent Variables

After we have finished data integration and our combined dataset is stored in `final_dataset`, the next step before modeling is to separate

- Independent variables (features) i.e. X
- Dependent variable (target/label) i.e. y

In our honeybee dataset, the target variable is the hive health indicator which is the column `Healthy`.

```
[74]
✓ Os
# Target variable (Dependent Variable)
y = final_dataset['Healthy']

# Feature variables (Independent Variables)
X = final_dataset.drop('Healthy', axis=1)

print('Shape of X (features):', X.shape)
print('Shape of y (target):', y.shape)
```

Shape of X (features): (35656, 66)
Shape of y (target): (35656,)

5.10 Data Reduction - Feature Selection and PCA

To reduce the feature space before modelling, three selection methods were applied to the full numeric feature set of 58 columns:

Correlation-based filtering: features with absolute correlation greater than 0.05 with the target variable were retained.

16.2 Correlation-Based Feature Selection

This works on numeric features so that we can select features with $|\text{correlation}| > 0.05$ with the target.

```
[77]
0s
# Combine X and y for correlation analysis
feature_data = X_numeric.copy()
feature_data['Healthy'] = y.values

numeric_feature_data = feature_data.select_dtypes(include=['int64', 'float64'])

# Correlation with target
target_corr = numeric_feature_data.corr()['Healthy'].drop('Healthy').sort_values(
    key=abs, ascending=False
)

print('Feature correlation with Healthy:')
print(target_corr.to_string())
```

Feature correlation with Healthy:

Healthy_encoded	1.000000e+00
Percent_Met	6.646322e-01
Stressors	6.399116e-01
Hive_Stress_Index	4.262491e-01
Apiary	3.882711e-01
Brood	3.830272e-01
Queen	3.669157e-01
InpsectionID	3.436116e-01
Colony_Risk_Score	3.433970e-01
Hive_Tag	-3.092374e-01
State	2.688802e-01
Wind_Stress	1.331468e-01
City	1.274780e-01
ObsID	-1.061333e-01
Precip_Total	1.039121e-01
Humidity_Min	-1.004465e-01
Cluster	-8.794828e-02
Humidity_Mean	-8.659901e-02
Wind_Direction	-6.884589e-02
Station	6.465900e-02
Rain_Stress	5.781923e-02
Temp_Range	5.367234e-02
Month	-4.986088e-02
Pressure_Mean	-4.820029e-02
Humidity_Max	-4.630915e-02
Humidity	-4.561195e-02
Dew_Point	-4.180287e-02
Humidity_Smoothed	-3.789700e-02
WeatherID	-3.271769e-02
Humidity_Stress	-3.095128e-02
Weather Stress Index	2.803656e-02

ables Terminal 4:27

Weather_Stress_Index	2.803656e-02
Condition	2.765195e-02
StationID	2.498826e-02
Temp_Max	2.012086e-02
WindSpeed_Mean	1.690441e-02
Temp_Min	-1.404172e-02
Station_City	1.296730e-02
Pressure	1.085281e-02
Temp_Mean	-1.013558e-02
Pressure_Smoothed	9.636720e-03
Wind_Speed	9.313166e-03
Wind_Speed_Smoothed	8.242640e-03
Temperature_Smoothed	-7.707892e-03
Temp_Smoothed	-7.707892e-03
Temperature	-3.928024e-03
Temp_RollingAvg	-3.483794e-03
WindSpeed_Max	1.949274e-04
Obs_Hour	-1.497818e-15
Bees	NaN
Food	NaN
Space	NaN
Wind_Gust	NaN
Precip	NaN
Temp_Stress	NaN

Random Forest feature importance: the top 10 features were selected as the primary feature set.

16.3 Random Forest (Feature Importance)

```
[79]
✓ 1x # Keep only numeric columns for model-based feature importance
X_numeric_clean = X_numeric.fillna(0)

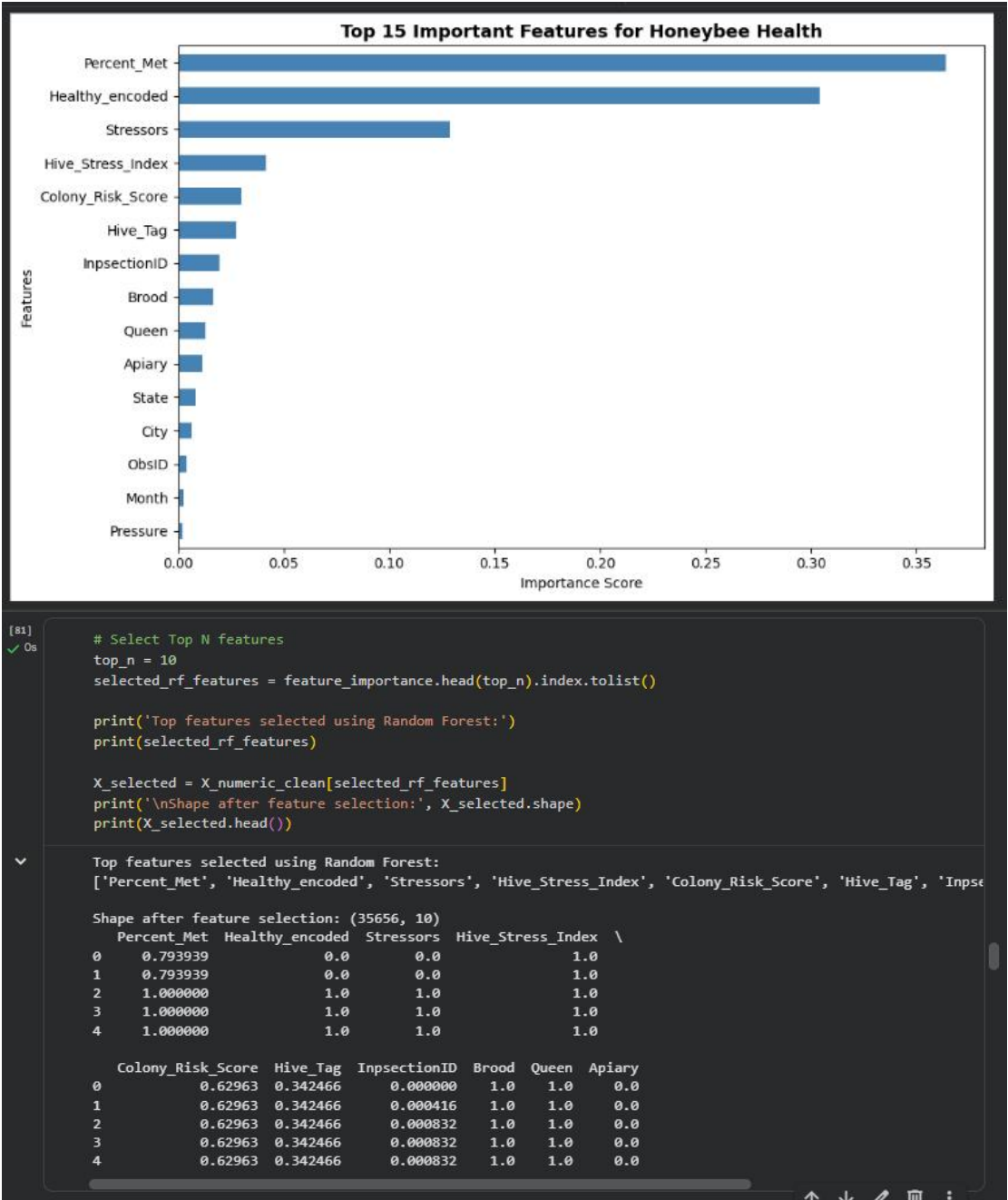
rf = RandomForestClassifier(random_state=42)
rf.fit(X_numeric_clean, y)

feature_importance = pd.Series(
    rf.feature_importances_,
    index=X_numeric_clean.columns
).sort_values(ascending=False)

print('Random Forest Feature Importance:')
print(feature_importance.to_string())
```

Random Forest Feature Importance:

Percent_Met	3.641683e-01
Healthy_encoded	3.041361e-01
Stressors	1.290621e-01
Hive_Stress_Index	4.151342e-02
Colony_Risk_Score	2.983348e-02
Hive_Tag	2.739959e-02
InspectionID	1.969823e-02
Brood	1.652352e-02
Queen	1.292360e-02
Apiary	1.137357e-02
State	8.127283e-03
City	6.311035e-03
ObsID	4.219477e-03
Month	2.498841e-03
Pressure	1.945574e-03
Cluster	1.908469e-03
Humidity_Mean	1.814984e-03
WindSpeed_Max	1.654051e-03
WindSpeed_Mean	1.577026e-03
Temp_Range	1.555944e-03
WeatherID	1.516728e-03
Temp_Min	1.426778e-03
Humidity_Min	1.217889e-03
Temp_Mean	1.162216e-03
Precip_Total	1.160799e-03
Temp_Max	1.090018e-03
Humidity_Max	1.020375e-03
Pressure_Mean	7.078638e-04
Dew_Point	6.292598e-04
Wind_Stress	5.599984e-04
Weather_Stress_Index	4.056053e-04
Humidity_Stress	3.100284e-04
Pressure_Smoothed	1.531742e-04
Temperature	8.807219e-05
Temp_RollingAvg	7.201153e-05
Wind_Speed_Smoothed	4.740655e-05
Wind_Direction	4.728531e-05
StationID	3.817236e-05
Rain_Stress	2.690404e-05
Humidity	2.185635e-05
Obs_Hour	1.844065e-05
Condition	1.229940e-05
Wind_Speed	1.208223e-05
Humidity_Smoothed	6.834353e-06
Station_City	6.300895e-06
Temp_Smoothed	3.434950e-06
Temperature_Smoothed	1.865354e-06
Station	1.191315e-07
Bees	0.000000e+00
Food	0.000000e+00
Wind_Gust	0.000000e+00
Space	0.000000e+00
Precip	0.000000e+00
Temp_Stress	0.000000e+00



SelectKBest with ANOVA F-score: select the top 10 features identified by statistical significance were used as a secondary comparison.

16.4 SelectKBest (Statistical Feature Selection)

Here, we are using ANOVA F-score because our target is a classification problem. This is mainly used to refer to a statistical feature selection technique.

```
[82]
✓ Os
# Impute any remaining NaN before SelectKBest
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X_numeric_clean)

k = min(10, X_numeric_clean.shape[1]) # ensure k <= number of features
selector = SelectKBest(score_func=f_classif, k=k)
X_kbest = selector.fit_transform(X_imputed, y)

selected_kbest_features = X_numeric_clean.columns[selector.get_support()].tolist()

print(f'Top {k} features selected using SelectKBest (ANOVA F-score):')
print(selected_kbest_features)
```

Top 10 features selected using SelectKBest (ANOVA F-score):
['InpsectionID', 'Brood', 'Queen', 'Stressors', 'Percent_Met', 'Hive_Tag', 'Apiary', 'Hive_Stress_Index', 'C

Principal Component Analysis (PCA) was also applied after StandardScaler normalization. The cumulative explained variance plot was used to automatically select the number of components retaining 90% of the total variance, resulting in 16 principal components (confirmed from notebook: Shape after PCA: (35,656, 16)).

```
[84]
✓ Os
pca = PCA()
X_pca_all = pca.fit_transform(X_scaled)

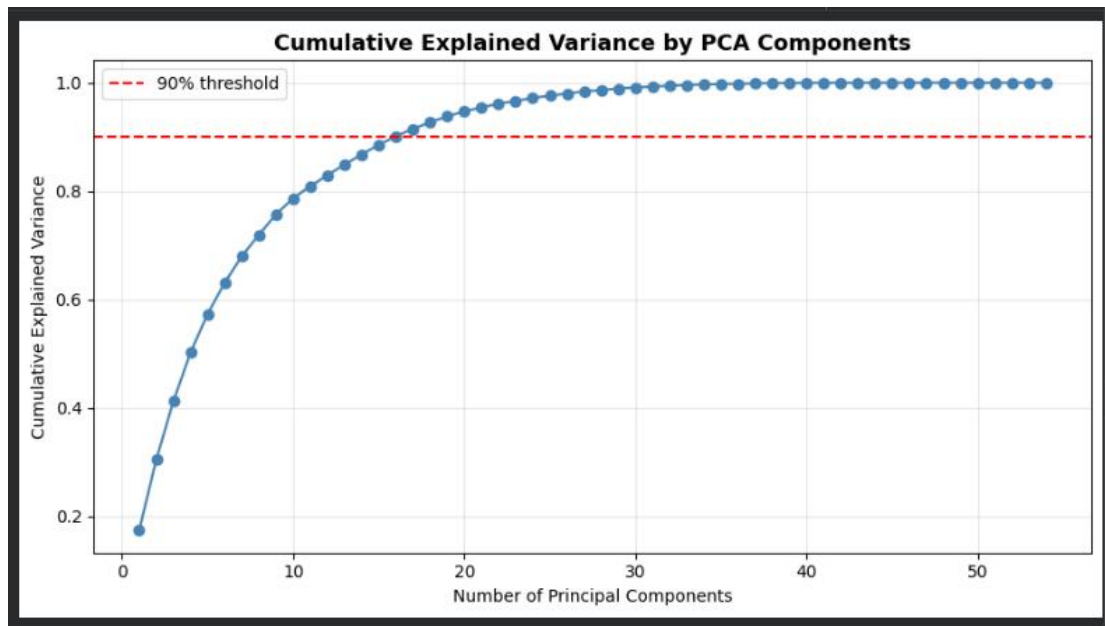
explained_variance = pca.explained_variance_ratio_
cumulative_variance = np.cumsum(explained_variance)

print('Explained variance ratio:')
print(explained_variance)

print('\nCummulative explained variance:')
print(cumulative_variance)
```

... Explained variance ratio:
[1.74135175e-01 1.30702678e-01 1.07486543e-01 8.98309616e-02
7.11885615e-02 5.79363706e-02 4.87468329e-02 3.95456102e-02
3.77182353e-02 2.97700403e-02 2.19025200e-02 2.04054760e-02
1.96046700e-02 1.86586075e-02 1.76034331e-02 1.60467929e-02
1.37854259e-02 1.16998027e-02 1.15097842e-02 8.50768666e-03
7.78759272e-03 6.49076935e-03 5.39110059e-03 5.29473962e-03
4.32213417e-03 3.93606406e-03 3.81100744e-03 2.81864971e-03
2.50649119e-03 2.14276232e-03 1.65158440e-03 1.44732021e-03
1.34779415e-03 1.10948626e-03 7.15540434e-04 6.16916704e-04
5.71823845e-04 4.56313358e-04 2.80831322e-04 2.35762875e-04
1.33599832e-04 7.93777817e-05 6.71303438e-05 7.40509862e-17
3.58529759e-17 1.63247121e-17 6.60961706e-18 1.05122465e-18
1.70159232e-33 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00]

Cummulative explained variance:
[0.17413518 0.30483785 0.4123244 0.50215536 0.57334392 0.63128029
0.68002712 0.71957273 0.75729097 0.78706101 0.80896353 0.829369
0.84897367 0.86763228 0.88523571 0.90128251 0.91506793 0.92676774
0.93827752 0.94678521 0.9545728 0.96106357 0.96645467 0.97174941
0.97607154 0.98000761 0.98381862 0.98663726 0.98914376 0.99128652
0.9929381 0.99438542 0.99573322 0.9968427 0.99755824 0.99817516
0.99874698 0.9992033 0.99948413 0.99971989 0.99985349 0.99993287
1. 1. 1. 1. 1. 1.
1. 1. 1. 1. 1. 1.]



16.6 Practically Logical Choice of Features

```
[89] ✓ 0s  
print('X_selected - Feature Selection result:')  
print(X_selected.shape)  
print(X_selected.head())
```

X_selected - Feature Selection result:
(35656, 10)

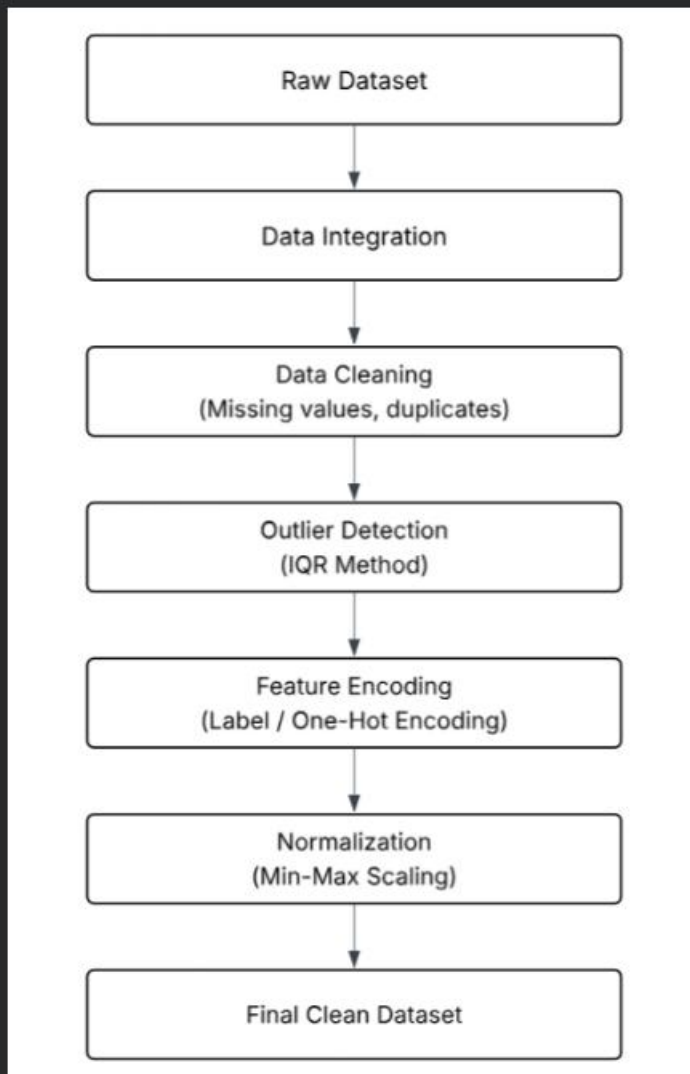
	Percent_Met	Healthy_encoded	Stressors	Hive_Stress_Index	\
0	0.793939	0.0	0.0	1.0	
1	0.793939	0.0	0.0	1.0	
2	1.000000	1.0	1.0	1.0	
3	1.000000	1.0	1.0	1.0	
4	1.000000	1.0	1.0	1.0	

	Colony_Risk_Score	Hive_Tag	InpsectionID	Brood	Queen	Apiary
0	0.62963	0.342466	0.000000	1.0	1.0	0.0
1	0.62963	0.342466	0.000416	1.0	1.0	0.0
2	0.62963	0.342466	0.000832	1.0	1.0	0.0
3	0.62963	0.342466	0.000832	1.0	1.0	0.0
4	0.62963	0.342466	0.000832	1.0	1.0	0.0

```
[90] ✓ 0s  
print('X_pca - PCA result:')  
print(X_pca.shape)
```

X_pca - PCA result:
(35656, 16)

18. Preprocessing Pipeline Flowchart



6. Data Modelling - Phase 2

6.1 Modelling Dataset Design

All modelling was performed on an inspection-level dataset of 2,404 rows - one row per inspection - using daily-aggregated weather features. Working at this level prevents data leakage that would occur if models were trained on the 35,656-row merged dataset, where each inspection repeats approximately 15 times due to the hourly weather join.

The column `Percent_Met` was excluded from features because it is directly derived from the target label - `Percent_Met` equals 100 if and only if `Healthy` equals Yes. Including it would allow the model to simply copy the target, not learn from real features. ID columns (`InpsectionID`, `HiveID`, `ApiaryID`, `WeatherID`, `ObsID`, `StationID`) were also excluded as non-predictive identifiers.

The final feature set contains 26 columns: the six binary inspection attributes (`Brood`, `Bees`, `Queen`, `Food`, `Stressors`, `Space`), ten daily weather aggregate columns

(Temp_Mean, Temp_Min, Temp_Max, Humidity_Mean, Humidity_Min, Humidity_Max, WindSpeed_Mean, WindSpeed_Max, Pressure_Mean, Precip_Total), four engineered stress features (Hive_Stress_Index, Weather_Stress_Index, Colony_Risk_Score, Temp_Range), three temporal features (Inspection_Month, Inspection_Year, Inspection_DayOfWeek), and three encoded location features (State_enc, City_enc, Apiary_enc).

6.2 Train/Test Split

The 2,404-row dataset was split into 80% training and 20% testing using stratified sampling to maintain the class ratio in both sets. The exact split from the notebook:

```
Training set : 1923 samples
Testing set  : 481 samples

Training class distribution:
  Healthy (1): 698 | Unhealthy (0): 1225

Testing class distribution:
  Healthy (1): 175 | Unhealthy (0): 306
```

6.3 Handling Class Imbalance - SMOTE

The training set had a class imbalance of 63.7% unhealthy vs 36.3% healthy. SMOTE (Synthetic Minority Oversampling Technique) was applied to the training set only to generate synthetic samples for the healthy minority class. The test set was left unchanged to reflect the real-world distribution. The exact results from the notebook:

```
Before SMOTE – Train class distribution:
  Healthy (1): 698 | Unhealthy (0): 1225

After SMOTE – Train class distribution:
  Healthy (1): 1225 | Unhealthy (0): 1225

New training set size: 2450 samples
```

6.4 Models Trained

Seven classification models were defined and trained on the SMOTE-balanced training set:

- Logistic Regression - linear baseline (max_iter=1000, random_state=42)
- Decision Tree - rule-based model (max_depth=10, random_state=42)
- Random Forest - bagging ensemble (n_estimators=100, random_state=42)
- Gradient Boosting - sequential boosting ensemble (n_estimators=100, random_state=42)
- K-Nearest Neighbors - distance-based classifier (n_neighbors=5)
- Naive Bayes - probabilistic Gaussian classifier
- AdaBoost - adaptive boosting ensemble (n_estimators=100, random_state=42)

```

Total models to evaluate: 7
- Logistic Regression
- Decision Tree
- Random Forest
- Gradient Boosting
- K-Nearest Neighbors
- Naive Bayes
- AdaBoost

```

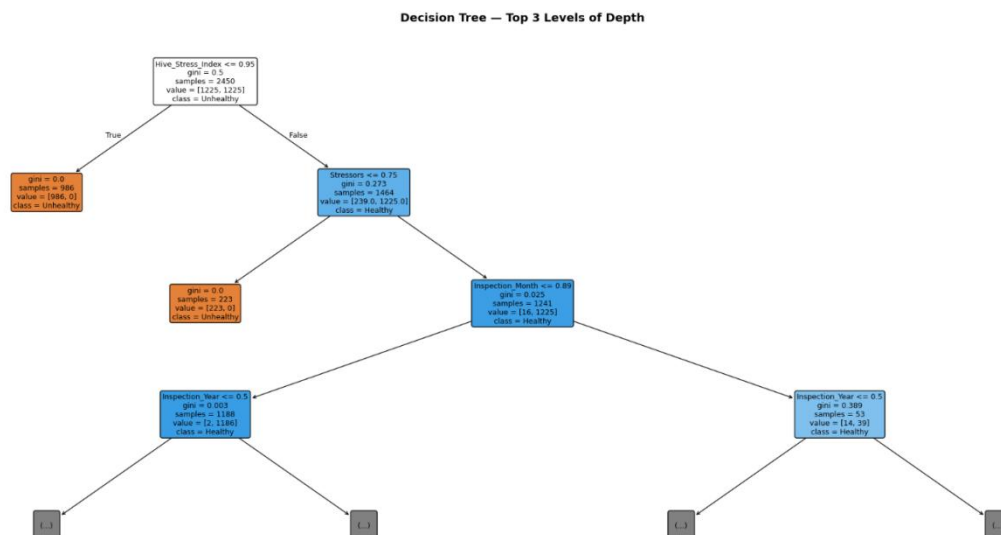
6.5 Decision Tree Visualization

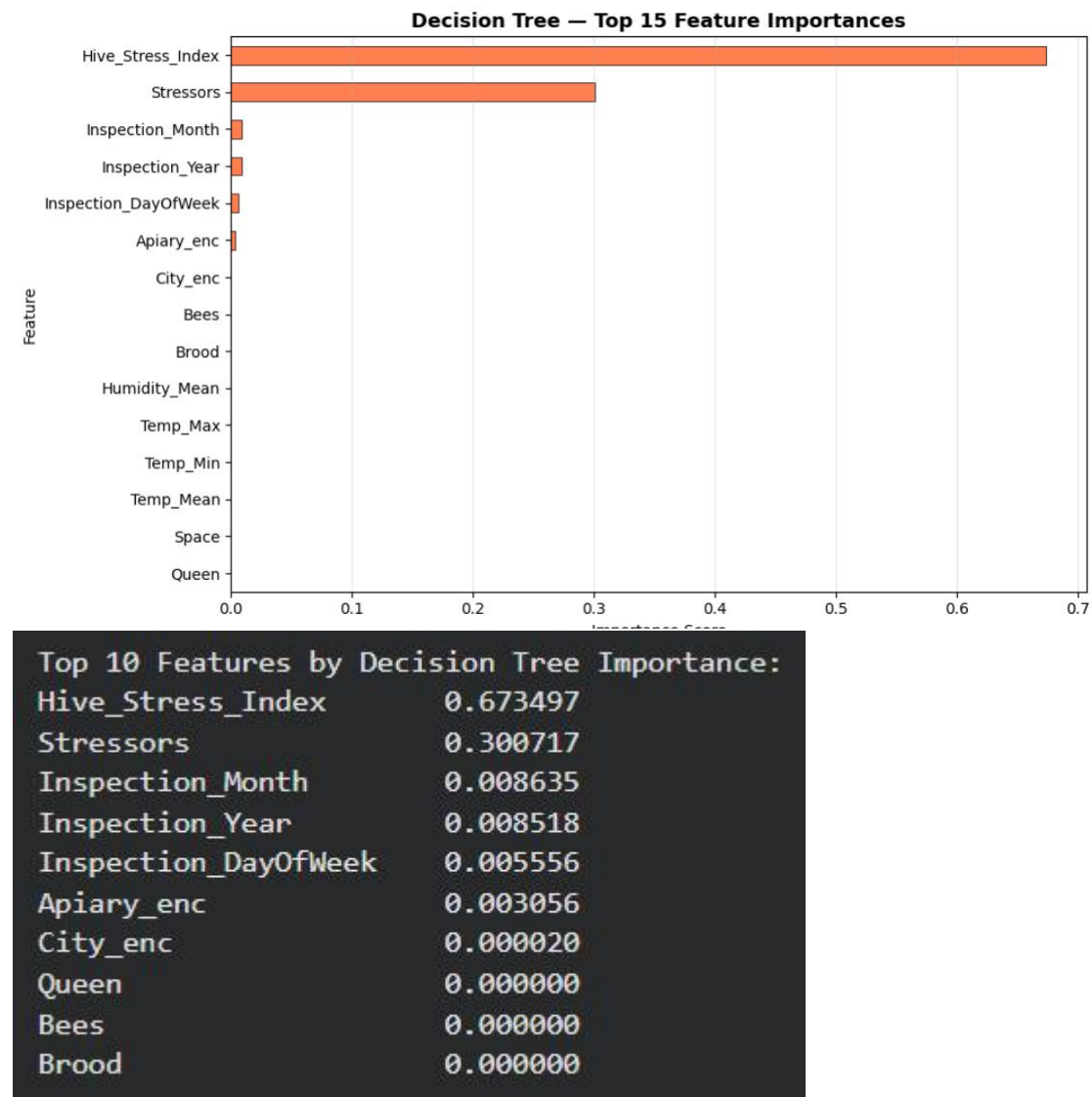
The Decision Tree model was additionally visualized to inspect the decision rules it learned. The text representation of the top 3 levels of the tree (from notebook output) shows:

```

|--- Hive_Stress_Index <= 0.95
|   |--- class: 0 (Unhealthy)
|--- Hive_Stress_Index > 0.95
|   |--- Stressors <= 0.75
|   |   |--- class: 0
|   |--- Stressors > 0.75
|       |--- Inspection_Month <= 0.89 → class: 1 (Healthy)
|       |--- Inspection_Month > 0.89 → class: 1 (Healthy)

```





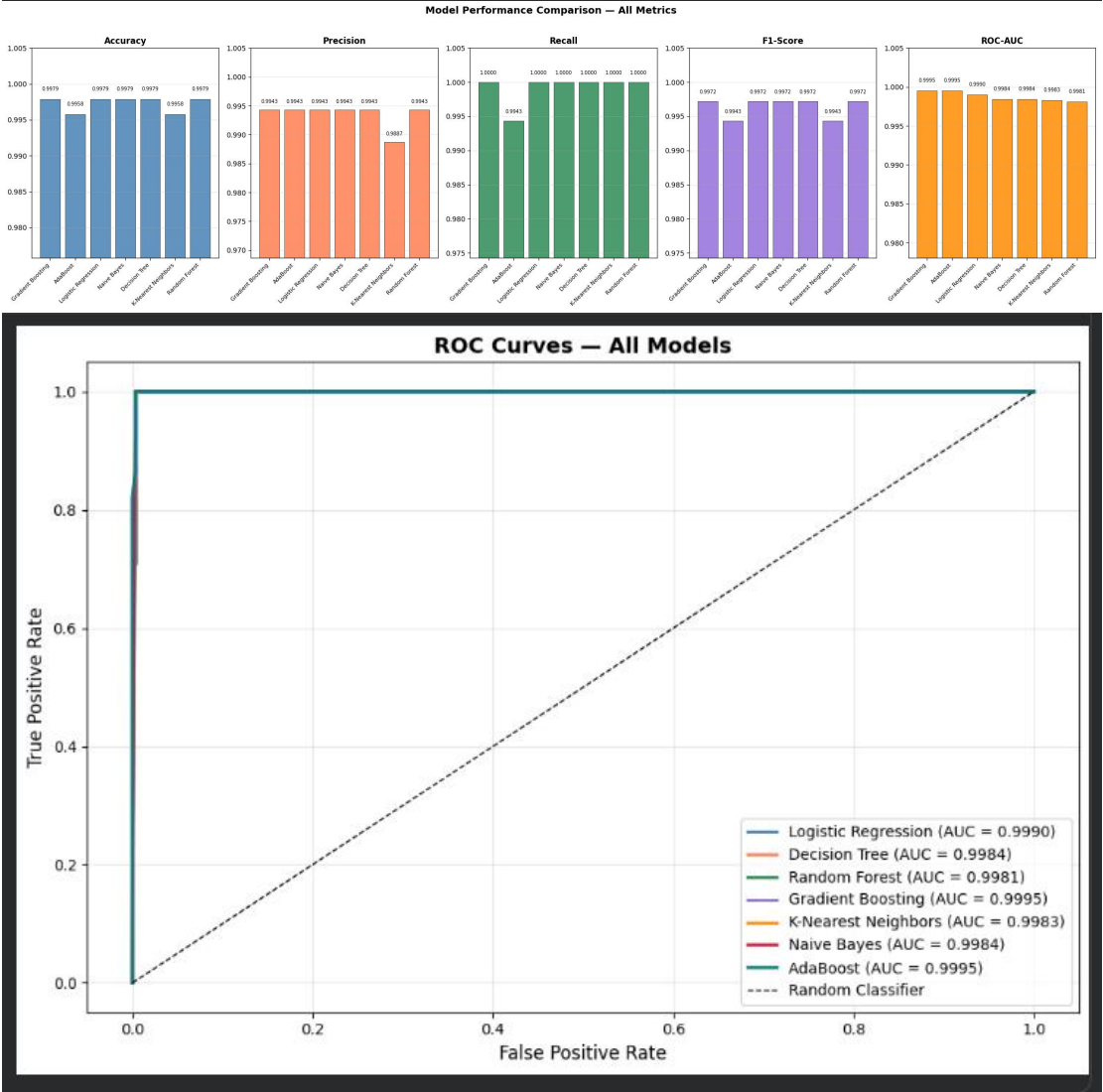
This shows that the tree primarily splits on `Hive_Stress_Index` at the first node, then uses `Stressors` at the second level. Colonies with a `Hive_Stress_Index` above 0.95 and `Stressors` above 0.75 are classified as healthy. The tree's own feature importance placed `Hive_Stress_Index` at 67.3% and `Stressors` at 30.1%.

6.6 Evaluation Metrics

All models were evaluated using five metrics on the same held-out test set of 481 samples:

- Accuracy: proportion of all predictions that are correct.
- Precision: proportion of predicted healthy that are actually healthy - minimizes false positives.
- Recall: proportion of actual healthy colonies correctly detected - minimizes false negatives.
- F1-Score: harmonic mean of precision and recall, most informative for imbalanced data.
- ROC-AUC: area under the ROC curve, measuring the model's ability to discriminate between classes across all classification thresholds. Used as the primary ranking metric.

MODEL COMPARISON TABLE (sorted by ROC-AUC)						
	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
1	Gradient Boosting	0.9979	0.9943	1.0000	0.9972	0.9995
2	AdaBoost	0.9958	0.9943	0.9943	0.9943	0.9995
3	Logistic Regression	0.9979	0.9943	1.0000	0.9972	0.9990
4	Naive Bayes	0.9979	0.9943	1.0000	0.9972	0.9984
5	Decision Tree	0.9979	0.9943	1.0000	0.9972	0.9984
6	K-Nearest Neighbors	0.9958	0.9887	1.0000	0.9943	0.9983
7	Random Forest	0.9979	0.9943	1.0000	0.9972	0.9981



7. Model Results and Comparison

The table below contains the exact results from the notebook output, sorted by ROC-AUC score.

Table 3: Model Comparison Table

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Gradient Boosting	0.9979	0.9943	1.0000	0.9972	0.9995

AdaBoost	0.9958	0.9943	0.9943	0.9943	0.9995
Logistic Regression	0.9979	0.9943	1.0000	0.9972	0.9990
Naive Bayes	0.9979	0.9943	1.0000	0.9972	0.9984
Decision Tree	0.9979	0.9943	1.0000	0.9972	0.9984
K-Nearest Neighbors	0.9958	0.9887	1.0000	0.9943	0.9983
Random Forest	0.9979	0.9943	1.0000	0.9972	0.9981

All seven models achieved very strong performance on this dataset, with ROC-AUC scores above 0.998 for every model. This indicates that the features engineered during preprocessing, particularly the Hive Stress Index and Colony Risk Score capture strong and clear signals that are highly predictive of colony health.

7.1 5-Fold Cross-Validation Results

To verify that performance was consistent and not due to a favorable train/test split, 5-fold stratified cross-validation was applied to the top three models. Results from the notebook:

Table 4: Cross-Validation Results

Model	Mean Accuracy	Std Deviation
Gradient Boosting	0.9979	0.0013
AdaBoost	0.9946	0.0021
Logistic Regression	0.9929	0.0041

Gradient Boosting achieved the lowest standard deviation (0.0013), confirming it is the most stable and consistent model. A lower standard deviation means the model performs reliably across different subsets of the data, not just on the specific train/test split used.

7.2 Observations on Individual Models

Gradient Boosting and AdaBoost both achieved the highest ROC-AUC of 0.9995. Gradient Boosting also achieved the lowest cross-validation variance, making it the overall best choice.

Logistic Regression achieved a strong ROC-AUC of 0.9990 for a linear model, suggesting the engineered features create a largely linearly separable problem.

Decision Tree achieved ROC-AUC of 0.9984 and provides the additional benefit of interpretable decision rules. Its top split on $\text{Hive_Stress_Index} \leq 0.95$ directly reflects the biological logic of colony health assessment.

Naive Bayes achieved ROC-AUC of 0.9984 despite making strong independence assumptions, showing the features are individually informative.

K-Nearest Neighbors achieved 0.9983 with slightly lower precision (0.9887) compared to other models.

Random Forest achieved 0.9981 - slightly lower than Gradient Boosting, which is expected since bagging is generally less precise than sequential boosting on structured tabular data.

8. Best Model Analysis

8.1 Best Model: Gradient Boosting

Gradient Boosting was identified as the best model. Exact metrics from the notebook:

```
=====
BEST MODEL: Gradient Boosting
=====
Accuracy : 0.9979
Precision : 0.9943
Recall    : 1.0000
F1-Score  : 0.9972
ROC-AUC   : 0.9995
=====

The Gradient Boosting achieved the highest ROC-AUC score,
making it the best model for predicting honeybee colony health.
```

The model correctly identified every single healthy colony in the test set (Recall = 1.0000). It misclassified only 1 out of 175 healthy test samples (Precision = 0.9943), and correctly classified all 306 unhealthy test samples. This means the model had zero false negatives on healthy colonies - a critical property in a real-world beekeeping application where missing a healthy colony is less costly than missing a stressed one.

Gradient Boosting works by building decision trees sequentially, where each new tree focuses on correcting the errors made by the previous tree. It uses a learning rate to control the contribution of each tree, which tends to produce more refined probability estimates compared to bagging-based methods like Random Forest. This explains both its higher ROC-AUC and lower cross-validation variance.

8.2 Feature Importance of the Best Model

The feature importance values from the Gradient Boosting best model (from notebook output):

Table 5: Feature Importance - Gradient Boosting (Top 10)

Rank	Feature	Importance Score / Description
1	Hive_Stress_Index	0.6796 - Composite of 5 inspection attributes
2	Stressors	0.3034 - Raw stressor presence from inspection

3	Inspection_Month	0.0104 - Month of inspection (seasonal signal)
4	Inspection_DayOfWeek	0.0029 - Day of week of inspection
5	Inspection_Year	0.0028 - Year of inspection
6	Apiary_enc	0.0004 - Encoded apiary location
7	Queen	0.0002 - Queen condition from inspection
8	Colony_Risk_Score	0.0001 - Combined hive + weather risk score
9	State_enc	0.0000 - Encoded state (NC or UT)
10	City_enc	0.0000 - Encoded city

The Hive_Stress_Index alone accounts for 67.96% of the model's decision-making, confirming that the engineered composite feature effectively summarizes the collective state of the colony. Stressors contributes a further 30.34%, indicating it carries strong independent information beyond what the Hive_Stress_Index captures. Together these two features account for over 98% of the model's importance weight. Weather and location features contribute minimally, suggesting that the internal colony state at inspection time is far more predictive than external environmental conditions on their own.

9. Hyperparameter Tuning

The models in Phase 2 were initially trained using default or manually chosen hyperparameters. Hyperparameter tuning was then applied to systematically search for better configurations. Two methods were used: GridSearchCV on Gradient Boosting and RandomizedSearchCV on Random Forest.

9.1 GridSearchCV - Gradient Boosting

GridSearchCV tests every combination in a defined parameter grid using 5-fold stratified cross-validation. The grid searched 16 combinations ($2 \times 2 \times 2 \times 2$), totalling 80 fits. Parameter grid used:

- n_estimators: [100, 200]
- max_depth: [3, 5]
- learning_rate: [0.05, 0.1]
- subsample: [0.8, 1.0]

Results from the notebook:

```
--- GridSearchCV Results (Gradient Boosting) ---
Best Parameters : {'learning_rate': 0.05, 'max_depth': 5, 'n_estimators': 200, 'subsample': 0.8}
Best CV ROC-AUC : 1.0000

Test Set Performance (Tuned Gradient Boosting):
Accuracy : 0.9979
Precision : 0.9943
Recall : 1.0000
F1-Score : 0.9972
ROC-AUC : 0.9997
```

9.2 RandomizedSearchCV - Random Forest

RandomizedSearchCV randomly samples 20 combinations from a parameter distribution, which is faster when the search space is large. It tested 20 combinations across 5 folds, totalling 100 fits. Distributions used:

- `n_estimators`: `randint(50, 300)`
- `max_depth`: `[None, 5, 10, 15, 20]`
- `min_samples_split`: `randint(2, 10)`
- `min_samples_leaf`: `randint(1, 5)`
- `max_features`: `['sqrt', 'log2']`

Results from the notebook:

```
--- RandomizedSearchCV Results (Random Forest) ---
Best Parameters : {'max_depth': None, 'max_features': 'log2', 'min_samples_leaf': 1, 'min_samples_split': 3, 'n_estimators': 133}
Best CV ROC-AUC : 1.0000

Test Set Performance (Tuned Random Forest):
Accuracy : 0.9979
Precision : 0.9943
Recall : 1.0000
F1-Score : 0.9972
ROC-AUC : 0.9980
```

9.3 Before vs After Tuning Comparison

Table 6: Default vs Tuned Model ROC-AUC on Test Set

Model	Default AUC	Tuned AUC
Gradient Boosting	0.9995	0.9997
Random Forest	0.9981	0.9980

```
=====
Comparison: Default vs Tuned Models (ROC-AUC on Test Set)
=====
Gradient Boosting (default) : 0.9995
Gradient Boosting (tuned)   : 0.9997
Random Forest   (default) : 0.9981
Random Forest   (tuned)   : 0.9980
```

GridSearchCV improved Gradient Boosting's test AUC from 0.9995 to 0.9997 - a small but measurable gain. The optimal configuration used a lower learning rate (0.05) with more estimators (200) and a maximum depth of 5, which is a standard pattern in gradient boosting where slower learning with more trees tends to generalize better. Random Forest's AUC remained essentially the same (0.9981 → 0.9980), which confirms that the default parameters were already near-optimal for this dataset.

10. Deep Learning Models

Two deep learning approaches were explored to compare against the classical classifiers and to investigate whether temporal patterns in the inspection sequences carry additional predictive information.

10.1 Feedforward Neural Network (FNN)

A standard feedforward neural network was built as a deep learning baseline. The architecture and total parameters from the notebook:

Input layer: 26 features

Dense(64, ReLU) → BatchNormalization → Dropout(0.3)

Dense(32, ReLU) → BatchNormalization → Dropout(0.2)

Dense(16, ReLU)

Dense(1, Sigmoid) - output

Total parameters: 4,737 (4,545 trainable, 192 non-trainable)

```
=====
1. Feedforward Neural Network (FNN)
=====
Architecture: Input → Dense(64) → BN → Dropout → Dense(32) → BN → Dropout → Dense(16) → Output
Model: "sequential_8"
```

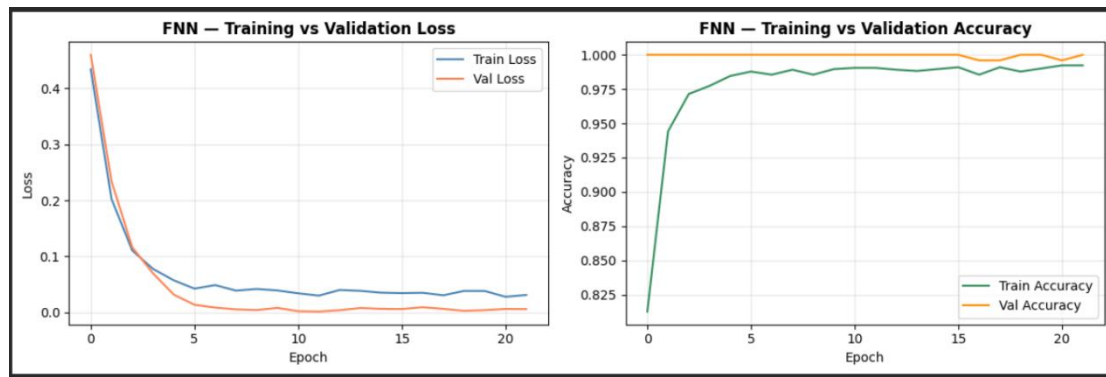
Layer (type)	Output Shape	Param #
dense_24 (Dense)	(None, 64)	1,728
batch_normalization_8 (BatchNormalization)	(None, 64)	256
dropout_16 (Dropout)	(None, 64)	0
dense_25 (Dense)	(None, 32)	2,080
batch_normalization_9 (BatchNormalization)	(None, 32)	128
dropout_17 (Dropout)	(None, 32)	0
dense_26 (Dense)	(None, 16)	528
dense_27 (Dense)	(None, 1)	17

```
Total params: 4,737 (18.50 KB)
Trainable params: 4,545 (17.75 KB)
Non-trainable params: 192 (768.00 B)
```

The model was compiled with Adam optimizer (learning_rate=0.001) and binary cross-entropy loss. Training used early stopping with patience=10 monitoring validation loss, and a 90/10 train/validation split within the SMOTE-balanced training set. The model stopped early before 50 epochs. Results from the notebook:

```
Feedforward Neural Network – Test Set Performance:
```

Accuracy	: 0.9979
Precision	: 0.9943
Recall	: 1.0000
F1-Score	: 0.9972
ROC-AUC	: 0.9975



The FNN matched the accuracy, precision, recall, and F1-score of the best classical models. Its ROC-AUC of 0.9975 is slightly below Gradient Boosting's 0.9995, which is expected since neural networks generally require larger datasets to fully utilize their capacity. The training curves showed rapid convergence - validation accuracy reached near 1.0 by Epoch 4.

10.2 LSTM - Time-Series Inspection Sequences per Hive

The LSTM model treats inspection records as temporal sequences per hive, processing the last 3 consecutive inspections for each hive as one input. This allows the model to learn how a colony's condition changes over time - something classical models cannot capture since they treat each inspection independently.

Sequence construction (from notebook):

```
=====
2. LSTM — Time-Series Inspection Sequences per Hive
=====
Total LSTM sequences : 2061
Sequence shape       : (2061, 3, 26) (samples, timesteps, features)
Class distribution    : Healthy=746 Unhealthy=1315

Train sequences (after SMOTE): 2102
Test sequences       : 413
```

LSTM architecture:

LSTM(64, return_sequences=True) - 23,296 parameters

Dropout(0.2)

LSTM(32) - 12,416 parameters

Dropout(0.2)

Dense(16, ReLU)

Dense(1, Sigmoid)

Total parameters: 36,257 (all trainable)

Model: "sequential_9"

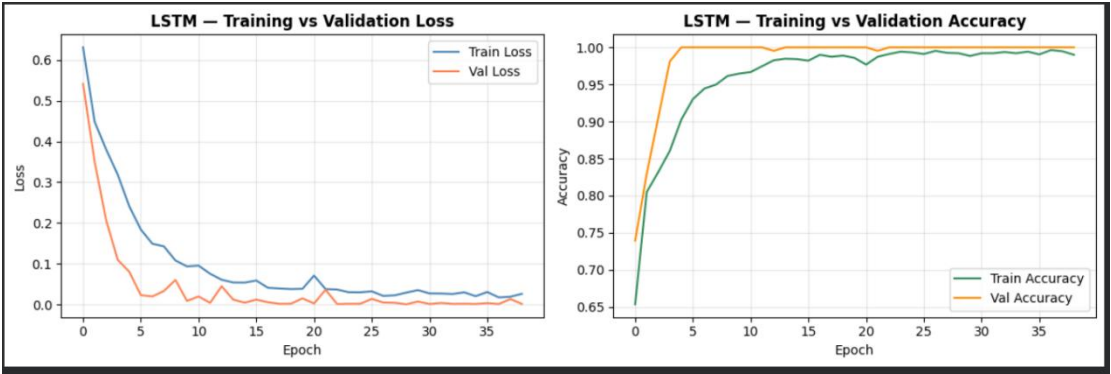
Layer (type)	Output Shape	Param #
lstm_8 (LSTM)	(None, 3, 64)	23,296
dropout_18 (Dropout)	(None, 3, 64)	0
lstm_9 (LSTM)	(None, 32)	12,416
dropout_19 (Dropout)	(None, 32)	0
dense_28 (Dense)	(None, 16)	528
dense_29 (Dense)	(None, 1)	17

Total params: 36,257 (141.63 KB)
Trainable params: 36,257 (141.63 KB)
Non-trainable params: 0 (0.00 B)

Training used Adam (learning_rate=0.001), binary cross-entropy, and early stopping with patience=10. Results from the notebook:

LSTM – Test Set Performance:

Accuracy	: 0.9709
Precision	: 0.9255
Recall	: 1.0000
F1-Score	: 0.9613
ROC-AUC	: 0.9991



10.3 Deep Learning Comparison Summary

Table 7: Deep Learning vs Best Classical Model

Model	Accuracy	Precision	Recall	F1	AUC
Gradient Boosting (classical)	0.9979	0.9943	1.0000	0.9972	0.9995
Feedforward NN (deep learning)	0.9979	0.9943	1.0000	0.9972	0.9975
LSTM (time-series)*	0.9709	0.9255	1.0000	0.9613	0.9991

*The LSTM uses a smaller, sequence-based test set (413 samples) so its metrics are not directly comparable to classical models evaluated on the full 481-sample test set.

```
=====
Deep Learning vs Best Classical Model (Gradient Boosting)
=====

Gradient Boosting (classical) AUC : 0.9995
Feedforward NN      (deep learning) AUC : 0.9975
LSTM                 (time-series)  AUC : 0.9991
**LSTM uses a smaller test set (sequence-based split) so scores
are not directly comparable with classical models on the full test set.
```

The FNN matched classical model performance almost exactly. The LSTM's lower accuracy and F1-score are partly due to the reduced test set size and the loss of some inspections when building sequences (hives with fewer than 3 inspections are excluded). Its ROC-AUC of 0.9991 remains very strong, and its perfect recall (1.0000) is notable. The LSTM demonstrated that temporal patterns in inspection sequences carry meaningful predictive signal, though the classical Gradient Boosting model remains the overall best performer on this dataset at its current size.

11. Key Findings

1. Colony health is mainly determined by hive inspection attributes. The Hive_Stress_Index (67.96%) and Stressors (30.34%) together account for over 98% of the Gradient Boosting model's feature importance, confirming that the internal state of the colony at inspection time is the dominant predictor.
2. Engineered composite features outperformed individual raw attributes. The Hive Stress Index and Colony Risk Score compressed domain knowledge into compact predictive signals that simplified the learning task for all models.
3. Weather conditions contributed through aggregated stress indices rather than raw hourly readings. The one-to-many join and subsequent daily aggregation were essential design decisions that significantly shaped the quality of weather-based features.
4. Class imbalance (63.7% unhealthy) was a critical issue. SMOTE was necessary to prevent models from being biased toward predicting the majority class. Without it, recall on healthy colonies would have been substantially lower.
5. All seven classical models achieved ROC-AUC above 0.998, suggesting the engineered features create a nearly separable classification problem. Gradient Boosting and AdaBoost jointly achieved the highest AUC of 0.9995.
6. Gradient Boosting was selected as the best model based on highest ROC-AUC (0.9995), highest cross-validation stability (std = 0.0013), and perfect recall (1.0000) - meaning it correctly identified every healthy colony in the test set.
7. Hyperparameter tuning slightly improved Gradient Boosting from AUC 0.9995 to 0.9997, with optimal parameters of learning_rate=0.05, n_estimators=200, max_depth=5, subsample=0.8.
8. The Feedforward Neural Network matched classical model performance (AUC = 0.9975), confirming the problem is well-suited to both linear and non-linear approaches.
9. The LSTM model demonstrated that temporal patterns in per-hive inspection sequences carry predictive value, achieving AUC = 0.9991 and perfect recall on the sequence-based test set.

10. Seasonal effects were present - colony health varied across months, with `Inspection_Month` ranking as the 3rd most important feature in the best model.

12. Future Work

12.1 Extended Hyperparameter Tuning

The current tuning used a limited grid and 20 random iterations due to computational constraints. A broader search using `RandomizedSearchCV` with more iterations, or Bayesian optimization, could further refine model performance. For Gradient Boosting specifically, additional parameters worth exploring include `min_samples_leaf`, `max_features`, and `warm_start`.

12.2 Deep Learning with Larger Sequence Windows

The LSTM was trained with a sequence length of 3. Using longer sequences (5 to 10 inspections) or bidirectional LSTMs would allow the model to capture longer-range temporal dependencies in colony health. Additionally, attention mechanisms could be incorporated to identify which inspection timesteps in a sequence are most predictive of the final health outcome.

12.3 Deployment as a Beekeeper Tool

The trained Gradient Boosting model could be packaged into a simple application where a beekeeper enters inspection observations and receives a health prediction with a confidence score. This would translate the academic results of this project into a practical tool for real-world use.

13. Conclusion

This project applied a complete two-phase data mining pipeline to a real-world honeybee health dataset. Phase 1 integrated six relational tables into a unified dataset, handled missing values and outliers, reduced noise in weather measurements using binning and smoothing, aggregated hourly weather into daily summaries, engineered biologically meaningful composite features, and prepared a clean 2,404-row inspection-level dataset for modelling. Phase 2 trained seven classification algorithms, compared their performance across five evaluation metrics, applied hyperparameter tuning, and explored deep learning approaches.

Gradient Boosting was identified as the best model with a test ROC-AUC of 0.9995, an accuracy of 99.79%, and a recall of 100% - meaning it correctly identified every healthy colony in the test set without missing any. Hyperparameter tuning further improved this to an AUC of 0.9997 using `learning_rate=0.05`, `n_estimators=200`, `max_depth=5`, and `subsample=0.8`. The Feedforward Neural Network matched classical model performance at AUC 0.9975, and the LSTM model demonstrated meaningful temporal patterns in per-hive inspection sequences at AUC 0.9991.

Feature importance analysis confirmed that hive inspection attributes - particularly the engineered Hive Stress Index (67.96%) and the raw Stressors column (30.34%) - are the dominant predictors. Weather conditions, temporal features, and location contributed a smaller but measurable share of the predictive signal.

The results demonstrate that data mining techniques can effectively predict honeybee colony health from inspection and weather records. A deployed version of the trained

model could assist beekeepers in identifying at-risk colonies earlier, reducing preventable losses and improving colony management decisions across beekeeping operations of all sizes.

14. Team Contribution

Component	Team Members Responsible
Data Quality Assessment & Cleaning	Nilam Rai, Bhavya Bhola
Data Integration	Jugias Girn, Bhavya Bhola
Noise Reduction & Feature Aggregation	Kenji Lopez, Bhavya Bhola
Feature Engineering	Bhavya Bhola, Kshitij Santokhee
Exploratory Data Analysis	Jugias Girn, Kenji Lopez, Bhavya Bhola
Feature Selection & PCA	Nilam Rai, Bhavya Bhola
Model Training & Evaluation	Kenji Lopez, Bhavya Bhola
Hyperparameter Tuning	Nilam Rai, Bhavya Bhola
Deep Learning (FNN & LSTM)	Kenji Lopez, Kshitij Santokhee
Report Writing & Documentation	All Members
Presentation	All Members

15. References

- [1] Redd, B. (2020). Predicting Honeybee Health from Hive and Weather Data. Kaggle. Retrieved from <https://www.kaggle.com/datasets/se18m502/bee-pollen-and-hive-activity>
- [2] Han, J., Kamber, M., & Pei, J. (2011). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann.
- [3] Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From Data Mining to Knowledge Discovery in Databases. *AI Magazine*, 17(3), 37–54.
- [4] Brownlee, J. (2020). Data Preparation for Machine Learning. *Machine Learning Mastery*.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [6] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- [7] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
- [8] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [9] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.

- [10] Genersch, E. (2010). Honey bee pathology: current threats to honey bees and beekeeping. *Applied Microbiology and Biotechnology*, 87(1), 87–97.
- [11] VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media.
- [12] McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.
- [13] Waskom, M. (2021). Seaborn: Statistical Data Visualization. *Journal of Open Source Software*, 6(60), 3021.
- [14] United States Department of Agriculture (USDA). (2023). *Honey Bee Colonies Report*. National Agricultural Statistics Service. Retrieved from <https://usda.gov>